☐ หน้าหลัก
1 Data Science คืออะไร? ›
2 การตั้งคำถามและกำหนดปัญหา ›
3 Python สำหรับ Data Science ›
4 การเก็บข้อมูลและ Pandas ›
5 การทำความสะอาดและแปลงข้อมูล ›
6 สถิติเชิงพรรณนา EDA และ Data Visual ›
7 แนะนำ Machine Learning ›
8 Regression — การทำนายค่าต่อเนื่อง ⌄
  ปัญหา Regression
  Simple Linear Regression
  Multiple Linear Regression
  Evaluation Metrics
  Regression Diagnostics
  การปรับปรุง Model
  Polynomial Regression: การจัดการความสัมพ
  สรุปท้ายบทที่ 8
  เอกสารอ้างอิงประจำบทที่ 8
  แบบฝึกหัดท้ายบทที่ 8
9 Classification — การจำแนกประเภท ›
10 Ensemble Methods ›
11 Clustering — การจัดกลุ่มข้อมูล ›
12 Model Validation และ Selection ›
13 Data Storytelling และการเขียนรายงาน ›
14 โครงงาน Data Science: End-to-End Pr ›
☐ อภิธานศัพท์
☐ ดรรชนี

บทที่ 8: Regression — การทำนายค่าต่อเนื่อง

ภาค 3: Machine Learning CLO: CLO2, CLO3 **สัปดาห์:** 10

☐ วัตถุประสงค์การเรียนรู้

เมื่อศึกษาบทนี้จบแล้ว นักศึกษาจะสามารถ:

1. อธิบายหลักการ Linear Regression ได้
2. สร้างโมเดล Simple และ Multiple Linear Regression ด้วย scikit-learn ได้
3. คำนวณและตีความ Evaluation Metrics (MAE, MSE, RMSE, R^2) ได้
4. วิเคราะห์ Residual Plots เพื่อตรวจสอบข้อสมมติได้
5. ปรับปรุง Model ด้วย Feature Selection และ Regularization ได้

ปัญหา Regression

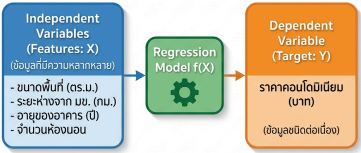
ในการศึกษาแขนงการเรียนรู้ของเครื่อง (Machine Learning) แบบมีผู้สอน (Supervised Learning) ปัญหาที่เราพบเจอสามารถแบ่งออกเป็นสองประเภทหลัก ได้แก่ การจำแนกประเภท (Classification) และการทำนายค่าต่อเนื่อง (Regression) ปัญหา Regression เป็นกระบวนการทางสถิติและการเรียนรู้ของเครื่องที่ใช้ในการประมาณความสัมพันธ์ระหว่างตัวแปรตาม (Dependent Variable หรือ Target) และตัวแปรอิสระ (Independent Variables หรือ Features) โดยมีจุดเด่นที่สำคัญคือ ตัวแปรตามในปัญหา Regression จะต้องเป็น **ข้อมูลชนิดต่อเนื่อง (Continuous Data)** (James et al., 2023)

ข้อมูลชนิดต่อเนื่องคือข้อมูลเชิงปริมาณที่สามารถมีค่าเป็นตัวเลขใดๆ ก็ได้ในช่วงที่กำหนด ซึ่งรวมถึงค่าทศนิยมและเศษส่วน ตัวอย่างเช่น ราคาของสินค้า น้ำหนัก ความสูง อุณหภูมิ หรือระยะเวลา ซึ่งแตกต่างจากปัญหา Classification ที่ตัวแปรตามจะเป็นข้อมูลชนิดไม่ต่อเนื่อง (Discrete Data) หรือข้อมูลเชิงหมวดหมู่ (Categorical Data) เช่น การทำนายว่าอีเมลเป็นสแปมหรือไม่ (ใช่/ไม่ใช่) หรือการทำนายเกรดของนักศึกษา (A, B, C, D, F)

ลักษณะของปัญหา Regression และเมื่อไหร่ที่ควรใช้

การตัดสินใจว่าปัญหาทางวิทยาการข้อมูลใดควรใช้เทคนิค Regression นั้น ให้พิจารณาจากคำถามที่ต้องการหาคำตอบ หากคำถามทางธุรกิจหรืองานวิจัยมีลักษณะเป็น “เท่าไหร่?” (How much? หรือ How many?) การใช้ Regression จะเป็นทางเลือกที่เหมาะสมที่สุด

พิจารณาองค์ประกอบของปัญหา Regression ดังแผนภาพด้านล่าง:



Regression Concept

ตารางเปรียบเทียบความแตกต่างระหว่าง Regression และ Classification

ลักษณะจำเพาะ	Regression	Classification
ชนิดของตัวแปรตาม (Target)	ข้อมูลชนิดต่อเนื่อง (Continuous)	ข้อมูลเชิงหมวดหมู่ (Categorical/Discrete)
ผลลัพธ์ของโมเดล	ตัวเลขจริง (Real numbers)	คลาส หรือ ความน่าจะเป็นของคลาส
ตัวอย่างคำถาม	บ้านหลังนี้ควรมีราคาเท่าไหร่?	ผู้ป่วยคนนี้เป็นโรคเบาหวานหรือไม่?
การวัดประสิทธิภาพ (Metrics)	MSE, RMSE, MAE, R-Squared	Accuracy, Precision, Recall, F1-Score

ตัวอย่างการประยุกต์ใช้ในบริบทประเทศไทย

เพื่อให้นักศึกษาสาขาวิทยาการคอมพิวเตอร์เห็นภาพการนำ Regression ไปประยุกต์ใช้ในการแก้ปัญหาจริง ต่อไปนี้คือตัวอย่างในบริบทของอุตสาหกรรมและสังคมไทย:

1. ด้านอสังหาริมทรัพย์ (Real Estate) การทำนายราคาที่อยู่อาศัยเป็นหนึ่งในปัญหา Regression ที่คลาสสิกที่สุด ตัวอย่างเช่น การสร้างโมเดลทำนายราคาเช่าหอพักหรือราคาขายคอนโดมิเนียมบริเวณรอบมหาวิทยาลัยขอนแก่น (โซนกังสดาล หรือ โซนหลัง มข.) * **Features (X):** ขนาดห้อง (ตารางเมตร), ระยะทางจากประตูมหาวิทยาลัย (กิโลเมตร), มีเครื่องปรับอากาศหรือไม่ (1=มี, 0=ไม่มี), อายุของอาคาร (ปี) * **Target (Y):** ราคาค่าเช่าต่อเดือน หรือ ราคาขาย (บาท)

2. ด้านทรัพยากรบุคคลและการศึกษา (Human Resources & Education) บริษัทเทคโนโลยีหรือธนาคารในประเทศไทยสามารถใช้ข้อมูลประวัติของนักศึกษาจบใหม่เพื่อทำนายอัตราเงินเดือนเริ่มต้น หรือทำนายระยะเวลาที่พนักงานจะทำงานกับองค์กร * **Features (X):** เกรดเฉลี่ยสะสม (GPAX), คะแนนสอบ TOEIC, จำนวนโปรเจกต์ที่เคยทำบน GitHub, ประสบการณ์ฝึกงาน (เดือน) * **Target (Y):** เงินเดือนเริ่มต้นที่เหมาะสม (บาท)

3. ด้านสาธารณสุขและสิ่งแวดล้อม (Public Health & Environment) ปัญหาฝุ่นละอองขนาดเล็ก (PM2.5) ในภาคตะวันออกเฉียงเหนือในช่วงฤดูแล้ง สามารถใช้โมเดล Regression ในการทำนายระดับความรุนแรงล่วงหน้าได้ * **Features (X):** อุณหภูมิ (องศาเซลเซียส), ความชื้นสัมพัทธ์ (%), ความเร็วลม (กิโลเมตร/ชั่วโมง), จำนวนจุดความร้อน (Hotspots) ในพื้นที่รัศมี 100 กิโลเมตร * **Target (Y):** ปริมาณฝุ่น PM2.5 (ไมโครกรัมต่อลูกบาศก์เมตร)

4. ด้านพาณิชย์อิเล็กทรอนิกส์ (E-commerce) แพลตฟอร์มซื้อขายออนไลน์สามารถทำนายยอดใช้จ่ายของลูกค้าแต่ละรายในแคมเปญส่งเสริมการขาย (เช่น 11.11 หรือ Payday) * **Features (X):** ระยะเวลาที่ใช้งานแอปพลิเคชัน (นาที), จำนวนครั้งที่คลิกดูสินค้า, ประวัติยอดใช้จ่ายเฉลี่ยในอดีต (บาท) * **Target (Y):** ยอดเงินสุทธิที่จะสั่งซื้อในแคมเปญนั้น (บาท)

รูปแบบทางคณิตศาสตร์เบื้องต้นของปัญหา Regression

ในทางสถิติและการเรียนรู้ของเครื่อง เราสมมติว่ามีความสัมพันธ์บางอย่างระหว่างตัวแปรอิสระ $X = (X_1, X_2, \ldots, X_p)$ และตัวแปรตาม Y ความสัมพันธ์นี้สามารถเขียนในรูปสมการทางคณิตศาสตร์ได้ดังนี้ (James et al., 2023):

$$Y = f(X) + \epsilon$$

โดยที่: * Y คือ ตัวแปรตาม (Target) ที่เราต้องการทำนาย * X คือ เวกเตอร์ของตัวแปรอิสระ (Features) * f คือ ฟังก์ชันที่แสดงความสัมพันธ์เชิงระบบ (Systematic information) ระหว่าง X และ Y ซึ่งเป็นสิ่งที่เราไม่ทราบค่าที่แท้จริง และอัลกอริทึม Machine Learning มีหน้าที่ในการ **ประมาณค่า (Estimate)** ฟังก์ชันนี้ (เรามักเขียนแทนด้วย $\hat{f}$) * ϵ (Epsilon) คือ ความคลาดเคลื่อนสุ่ม (Random Error Term) หรือ Irreducible Error ซึ่งครอบคลุมปัจจัยอื่นๆ ทั้งหมดที่ส่งผลต่อ Y แต่ไม่ได้ถูกรวมอยู่ใน X ค่าความคลาดเคลื่อนนี้มีค่าเฉลี่ยเป็นศูนย์

ทำไมเราถึงต้องประมาณค่า f? การประมาณค่า f มีจุดประสงค์หลัก 2 ประการ ได้แก่: 1. **การทำนาย (Prediction):** เมื่อเรามีข้อมูลใหม่ (X_{new}) เราต้องการทำนายค่า Y ที่จะเกิดขึ้น โดยใช้สมการ $\hat{Y} = \hat{f}(X_{new})$ เช่น การประเมินราคาบ้านหลังใหม่ที่กำลังจะสร้าง 2. **การอนุมาน (Inference):** เพื่อทำความเข้าใจความสัมพันธ์ระหว่าง X และ Y เช่น ต้องการทราบว่า หากระยะห่างจากมหาวิทยาลัยเพิ่มขึ้น 1 กิโลเมตร ราคาค่าเช่าหอพักจะลดลงเฉลี่ยกี่บาท ซึ่งมีประโยชน์อย่างมากในการวิเคราะห์เชิงนโยบาย

การสำรวจข้อมูลสำหรับปัญหา Regression ผ่านชุดข้อมูลจำลอง

ก่อนที่จะดำเนินการสร้างโมเดล Regression ใดๆ ขั้นตอนสำคัญคือการสำรวจข้อมูล (Exploratory Data Analysis - EDA) เพื่อทำความเข้าใจความสัมพันธ์ระหว่างตัวแปรอิสระและตัวแปรตาม (VanderPlas, 2016)

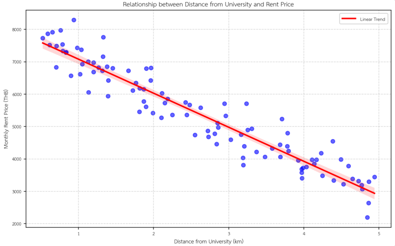
ตัวอย่างโค้ดภาษาไพธอน (Python) ด้านล่างนี้ เป็นการจำลองข้อมูลชุดหนึ่งที่แสดงความสัมพันธ์ระหว่าง “ระยะห่างจากมหาวิทยาลัยขอนแก่น” (Feature) และ “ราคาค่าเช่าหอพักต่อเดือน” (Target) พร้อมทั้งแสดงการสร้างกราฟเพื่อวิเคราะห์ความสัมพันธ์เบื้องต้น

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ตั้งค่า Seed เพื่อให้การสุ่มได้ผลลัพธ์เหมือนเดิมทุกครั้งที่รัน
np.random.seed(42)

# 1. การจำลองข้อมูล (Simulating Data)
# สมมติให้มีหอพักจำนวน 100 แห่ง
n_samples = 100

# Feature (X): ระยะห่างจาก มข. (กิโลเมตร) สุ่มค่าระหว่าง 0.5 ถึง 5.0 กม.
distance_km = np.random.uniform(0.5, 5.0, n_samples)

# Target (Y): ราคาค่าเช่าต่อเดือน (บาท)
# สมมติฐาน: ยิ่งไกล ราคาจะยิ่งถูกลง (ความสัมพันธ์เชิงลบ)
# ค่าเช่าพื้นฐาน 8000 บาท ลดลง 1000 บาทต่อระยะทาง 1 กม.
# เพิ่ม noise (epsilon) เพื่อจำลองความคลาดเคลื่อนในโลกจริง เช่น สภาพห้องต่างกัน
noise = np.random.normal(0, 500, n_samples) # Mean=0, SD=500
rent_price = 8000 - (1000 * distance_km) + noise

# สร้าง Pandas DataFrame เพื่อให้จัดการข้อมูลได้ง่าย (McKinney, 2022)
df_dorm = pd.DataFrame({
    'Distance_KM': distance_km,
    'Rent_Price_THB': rent_price
})

# 2. การแสดงผลข้อมูลเบื้องต้น
print("ตัวอย่างข้อมูล 5 บรรทัดแรก:")
print(df_dorm.head())
print("ข้อมูลสถิติเชิงพรรณนา:")
print(df_dorm.describe())

# 3. การสร้างกราฟแสดงความสัมพันธ์ (Scatter Plot with Regression Line)
# ใช้ Seaborn ช่วยในการสร้างกราฟที่สวยงามและมีเส้นแนวโน้ม (Trend line)
plt.figure(figsize=(10, 6))
sns.regplot(
    x='Distance_KM',
    y='Rent_Price_THB',
    data=df_dorm,
    scatter_kws={'alpha':0.6, 'color':'blue'}, # กำหนดจุดให้โปร่งแสง
    line_kws={'color':'red', 'label':'Linear Trend'} # กำหนดเส้นแนวโน้มสีแดง
)

plt.title('ความสัมพันธ์ระหว่างระยะห่างจากมหาวิทยาลัยและราคาค่าเช่าหอพัก', fontsize=14)
plt.xlabel('ระยะห่างจากมหาวิทยาลัย (กิโลเมตร)', fontsize=12)
plt.ylabel('ราคาค่าเช่าต่อเดือน (บาท)', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```

ผลลัพธ์:

```
ตัวอย่างข้อมูล 5 บรรทัดแรก:
   Distance_KM  Rent_Price_THB
0     2.185431     5858.092999
1     4.778214     3072.281946
2     3.793973     4251.907650
3     3.193963     3812.252364
4     1.202084     6688.080174

ข้อมูลสถิติเชิงพรรณนา:
       Distance_KM  Rent_Price_THB
count   100.000000      100.000000
```

```
mean      2.615813      5383.646475
std       1.338702      1477.697049
min       0.524850      2195.270244
25%       1.369403      4063.665714
50%       2.588641      5359.759445
75%       3.785914      6730.496331
max       4.940991      8292.096543
```



EDA Scatter Plot

จากการสำรวจข้อมูลด้วยแผนภาพการกระจาย (Scatter Plot) ข้างต้น หากเราพิจารณาจุดข้อมูล (สีน้ำเงิน) จะเห็นได้ว่าข้อมูลมีแนวโน้มลดลงจากซ้ายไปขวา ซึ่งสะท้อนให้เห็นถึงความสัมพันธ์เชิงลบ (Negative Correlation) ระหว่างระยะทางและราคาค่าเช่า นอกจากนี้ เส้นแนวโน้ม (เส้นสีแดง) ที่เกิดจากการลากผ่านจุดกึ่งกลางของกลุ่มข้อมูล คือตัวแทนของการประมาณค่าฟังก์ชัน $\hat{f}(X)$ อย่างง่ายด้วยสมการเชิงเส้น (Linear Regression) ซึ่งเราจะศึกษาอัลกอริทึมในการสร้างเส้นดังกล่าวอย่างละเอียดในหัวข้อถัดไป

ส่วนระยะห่างระหว่างจุดข้อมูลแต่ละจุดกับเส้นสีแดงในแนวดิ่ง สะท้อนถึงค่าความคลาดเคลื่อน ϵ (Error) ซึ่งในความเป็นจริงอาจเกิดจากปัจจัยอื่นๆ ที่เราไม่ได้นำมาพิจารณาในแกน X เช่น หอพักที่อยู่ไกลแต่อาจจะมีสระว่ายน้ำ หรือหอพักที่อยู่ใกล้แต่เป็นอาคารเก่า ทำให้ราคาจริงไม่ตกลงบนเส้นแนวโน้มเป๊ะตามทฤษฎี

สรุปประเด็นสำคัญ

- **ปัญหา Regression** คือกระบวนการเรียนรู้ของเครื่องแบบมีผู้สอน (Supervised Learning) ที่ใช้สำหรับทำนายตัวแปรตาม (Target) ที่เป็น **ข้อมูลชนิดต่อเนื่อง (Continuous Data)**
- ความแตกต่างที่ชัดเจนระหว่าง Regression และ Classification คือผลลัพธ์ที่ได้ หากต้องการทำนายปริมาณ (เช่น ราคา, อุณหภูมิ, ยอดขาย) จะใช้ Regression แต่หากต้องการทำนายหมวดหมู่ (เช่น ใช่/ไม่ใช่, ชนิดของโรค) จะใช้ Classification
- สมการพื้นฐานของ Regression คือ $Y = f(X) + \epsilon$ โดยเป้าหมายของ Machine Learning คือการหาวิธีประมาณค่าฟังก์ชัน f ให้มีความแม่นยำสูงสุด เพื่อใช้ในการทำนาย (Prediction) และการอนุมาน (Inference)
- ค่า ϵ (Irreducible Error) คือความคลาดเคลื่อนที่ไม่สามารถหลีกเลี่ยงได้ ซึ่งเกิดจากตัวแปรแฝงอื่นๆ ที่ไม่ได้ถูกเก็บข้อมูลมาวิเคราะห์

คำถามท้าย Section

1. สมมติว่านักศึกษาได้รับมอบหมายให้ทำโปรเจกต์วิเคราะห์ข้อมูลของโรงพยาบาลศรีนครินทร์ ระบบที่ต้องการสร้างคือระบบที่รับข้อมูลอายุ น้ำหนัก ความดันโลหิต และผลเลือดของผู้ป่วย เพื่อประเมินว่า “ผู้ป่วยรายนี้มีจำนวนวันที่คาดว่าจะต้องนอนพักรักษาตัวในโรงพยาบาล (Length of Stay) กี่วัน” ปัญหานี้จัดเป็นปัญหา Regression หรือ Classification? จงอธิบายเหตุผลประกอบ
2. จากสมการ $Y = f(X) + \epsilon$ หากนักศึกษาต้องการสร้างโมเดลทำนาย “เงินเดือนเริ่มต้นของบัณฑิตสาขาวิทยาการคอมพิวเตอร์” นักศึกษาคิดว่าค่า ϵ (ความคลาดเคลื่อนสุ่ม) ในบริบทนี้อาจเกิดจากปัจจัยใดได้บ้างที่มักจะไม่ถูกบันทึกไว้ในฐานข้อมูลผลการเรียนของมหาวิทยาลัย? จงยกตัวอย่างมา 2-3 ปัจจัย

Simple Linear Regression

Simple Linear Regression หรือ การวิเคราะห์การถดถอยเชิงเส้นอย่างง่าย เป็นหนึ่งในเทคนิคพื้นฐานที่สำคัญที่สุดในสถิติศาสตร์และ Machine Learning (ประเภท Supervised Learning) ซึ่งใช้สำหรับวิเคราะห์และสร้างแบบจำลองความสัมพันธ์ระหว่างตัวแปรอิสระ (Independent Variable หรือ Feature) จำนวนหนึ่งตัว และตัวแปรตาม (Dependent Variable หรือ Target) ที่มีลักษณะเป็นข้อมูลเชิงปริมาณที่มีค่าต่อเนื่อง (Continuous Data) (James et al., 2023)

จุดประสงค์หลักของ Simple Linear Regression คือการค้นหา “เส้นตรง” ที่สามารถอธิบายความสัมพันธ์ระหว่างตัวแปรทั้งสองได้ดีที่สุด เพื่อนำเส้นตรงดังกล่าวไปใช้ในการตีความพฤติกรรมของข้อมูล หรือใช้สำหรับทำนาย (Predict) ค่าของตัวแปรตามเมื่อทราบค่าของตัวแปรอิสระ

รูปแบบของสมการคณิตศาสตร์ (Mathematical Equation)

ภายใต้สมมติฐานว่าตัวแปรอิสระ (X) และตัวแปรตาม (Y) มีความสัมพันธ์กันเชิงเส้นตรง ความสัมพันธ์ดังกล่าวสามารถเขียนให้อยู่ในรูปสมการทางคณิตศาสตร์ได้ดังนี้:

$Y = \beta_0 + \beta_1 X + \epsilon$

โดยที่องค์ประกอบแต่ละส่วนมีความหมายดังต่อไปนี้:

- **Y (Dependent Variable):** ตัวแปรตาม หรือตัวแปรที่เราต้องการทำนายผล
- **X (Independent Variable):** ตัวแปรอิสระ หรือตัวแปรที่ใช้เป็นข้อมูลนำเข้าเพื่ออธิบาย Y
- **β_0 (Intercept):** จุดตัดแกน Y เป็นค่าพารามิเตอร์ที่บ่งบอกว่า หากค่า X มีค่าเท่ากับศูนย์ ค่า Y จะมีค่าพื้นฐานอยู่ที่เท่าใด
- **β_1 (Slope):** ความชันของเส้นตรง เป็นค่าพารามิเตอร์ที่บ่งบอกถึงขนาดและทิศทางของความสัมพันธ์ โดยระบุว่า หากค่า X เปลี่ยนแปลงไป 1 หน่วย ค่า Y จะเปลี่ยนแปลงไปกี่หน่วยโดยเฉลี่ย
- **ϵ (Error Term):** ความคลาดเคลื่อนสุ่ม (Random Error) ซึ่งเป็นตัวแทนของปัจจัยอื่นๆ ทั้งหมดที่ส่งผลต่อ Y แต่ไม่ได้ถูกรวมอยู่ในตัวแปร X รวมถึงความคลาดเคลื่อนจากการวัด

ตัวอย่างการตีความในบริบทจริง: พิจารณาแพลตฟอร์ม E-commerce ในประเทศไทย สมมติว่าต้องการศึกษาความสัมพันธ์ระหว่างงบประมาณที่ใช้ยิงโฆษณาบนโซเชียลมีเดีย (ตัวแปร X, หน่วย: หมื่นบาท) และยอดขายสินค้าที่เกิดขึ้นในเดือนนั้น (ตัวแปร Y, หน่วย: หมื่นบาท) หากสมการที่ได้จากการวิเคราะห์คือ $Y = 50 + 3.5X$ สามารถตีความได้ดังนี้: * **β_0 = 50:** หากในเดือนนั้นแพลตฟอร์มไม่มีการใช้งบประมาณในการยิงโฆษณาเลย ($X = 0$) คาดการณ์ว่ายอดขายสินค้าจะอยู่ที่ 50 หมื่นบาท (500,000 บาท) ซึ่งอาจมาจากลูกค้าประจำหรือการค้นหาแบบออร์แกนิก * **β_1 = 3.5:** ทุกๆ การเพิ่มงบประมาณโฆษณา 1 หมื่นบาท ยอดขายสินค้าจะเพิ่มขึ้นโดยเฉลี่ย 3.5 หมื่นบาท (35,000 บาท)

วิธีกำลังสองน้อยที่สุด (Ordinary Least Squares: OLS)

ในทางปฏิบัติ เราไม่สามารถทราบค่าพารามิเตอร์ที่แท้จริงของประชากร (β_0 และ β_1) ได้ เราจึงต้องประมาณค่าพารามิเตอร์เหล่านี้จากชุดข้อมูลตัวอย่าง (Sample Data) โดยใช้สัญลักษณ์ $\hat{\beta}_0$ และ $\hat{\beta}_1$ ทำให้ได้สมการสำหรับทำนาย (Prediction Equation) ดังนี้:

$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$

เมื่อ $\hat{y}_i$ คือค่าที่ได้จากการทำนาย วิธีการที่ได้รับความนิยมสูงสุดในการหาค่า $\hat{\beta}_0$ และ $\hat{\beta}_1$ ที่ทำให้เส้นตรง “เหมาะสมที่สุด (Best Fit)” กับข้อมูล คือ **วิธีกำลังสองน้อยที่สุด (Ordinary Least Squares หรือ OLS)**

หลักการของ OLS คือการสร้างเส้นตรงที่ทำให้ผลรวมของกำลังสองของความคลาดเคลื่อน (Sum of Squared Residuals: SSR) มีค่าน้อยที่สุด

ความคลาดเคลื่อน หรือ ส่วนตกค้าง (Residual: e_i) คือความแตกต่างระหว่างค่าจริง (y_i) และค่าที่ทำนายได้จากเส้นตรง ($\hat{y}_i$) สามารถเขียนเป็นสมการได้ว่า: $e_i = y_i - \hat{y}_i$

ฟังก์ชันเป้าหมาย (Objective Function) ของ OLS ที่อัลกอริทึมต้องหาค่าต่ำสุด (Minimize) คือ:

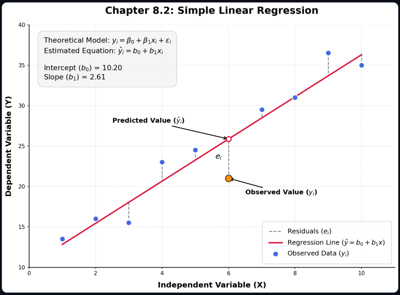
$$SSR = \sum_{i=1}^{n} e_i^2 = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = \sum_{i=1}^{n} (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2$$

สาเหตุที่ต้องยกกำลังสองค่า Residual (e_i^2) มีสองประการหลัก: 1. เพื่อป้องกันไม่ให้ค่าความคลาดเคลื่อนที่เป็นบวกและลบหักล้างกันเอง 2. เพื่อเป็นการลงโทษ (Penalize) ข้อมูลที่มีความคลาดเคลื่อนสูง (Outliers) ให้มีผลต่อการปรับเส้นตรงมากขึ้น

แผนภาพแสดงความสัมพันธ์ของ OLS:

```
        Y (ยอดขาย)
        |
        |          * (y_i: ข้อมูลจริง)
        |          |
        |          | ⟵ Residual (e_i = y_i - y_hat_i)
        |          |
        |----------o (y_hat_i: ค่าที่ทำนายได้บนเส้นตรง)
        |         /
        |        /     สมการเส้นตรง: Y_hat = β_0 + β_1*X
        |       /
        |      /
        |     /  ⟵ Intercept (β_0)
        |____/___________________________________ X (งบโฆษณา)
                   x_i
```

อธิบายแผนภาพ: จุด `*` คือจุดข้อมูลจริง ส่วนจุด `o` คือจุดที่อยู่บนเส้นการถดถอย ระยะห่างแนวตั้งระหว่าง `*` และ `o` คือค่า Residual อัลกอริทึม OLS จะพยายามปรับความชันและจุดตัดแกน Y เพื่อให้ผลรวมของพื้นที่สี่เหลี่ยมจัตุรัสที่เกิดจากระยะห่างเหล่านี้ (Squared Residuals) มีขนาดเล็กที่สุด



Linear Regression Line

การสร้างแบบจำลองด้วย Python และการแสดงผล (Visualizing Scatter, Line, and Residuals)

ในการประยุกต์ใช้ Simple Linear Regression ด้วยภาษา Python ไลบรารีมาตรฐานที่นิยมใช้คือ `scikit-learn` (สำหรับการสร้างแบบจำลอง) ร่วมกับ `matplotlib` และ `numpy` (สำหรับการจัดการข้อมูลและแสดงผล) (VanderPlas, 2016)

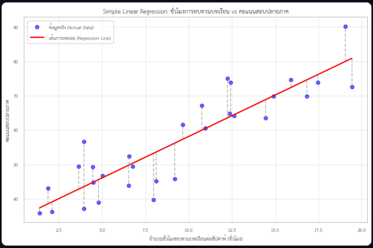
พิจารณาตัวอย่างการสร้างแบบจำลองทำนายคะแนนสอบปลายภาค (Final Score) ของนักศึกษาสาขาวิทยาการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น โดยใช้จำนวนชั่วโมงที่ใช้ในการทบทวนบทเรียนและเขียนโปรแกรมต่อสัปดาห์ (Study Hours) เป็นตัวแปรอิสระ

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# 1. การจำลองชุดข้อมูล (Simulating Data)
np.random.seed(42) # กำหนด seed เพื่อให้ผลลัพธ์คงที่
# ตัวแปรอิสระ X: จำนวนชั่วโมงที่ใช้ทบทวนบทเรียนต่อสัปดาห์ (1 ถึง 20 ชั่วโมง) จำนวน 30 คน
X = np.random.uniform(1, 20, 30).reshape(-1, 1)

# ตัวแปรตาม y: คะแนนสอบปลายภาค (คะแนนเต็ม 100)
# สมมติให้สมการจริงคือ y = 30 + 3*X + ความคลาดเคลื่อนสุ่ม(noise)
noise = np.random.normal(0, 8, 30) # ความคลาดเคลื่อนมีการแจกแจงแบบปกติ
y = 30 + 3 * X.flatten() + noise

# 2. การสร้างและฝึกสอนแบบจำลอง (Model Building & Training)
model = LinearRegression() # เรียกใช้ OLS Linear Regression
model.fit(X, y) # ฝึกสอนแบบจำลองด้วยข้อมูล X และ y

# 3. การดึงค่าพารามิเตอร์ที่แบบจำลองเรียนรู้ (Extracting Parameters)
beta_0 = model.intercept_
beta_1 = model.coef_[0]

print(f"สมการการถดถอยที่ได้: y_hat = {beta_0:.2f} + {beta_1:.2f}X")
```

```
# 4. การทำนายผล (Prediction)
y_pred = model.predict(X)

# 5. การแสดงผลด้วยกราฟ (Visualization)
plt.figure(figsize=(10, 6))

# พล็อตจุดข้อมูลจริง (Scatter Plot)
plt.scatter(X, y, color='blue', alpha=0.6, label='ข้อมูลจริง (Actual Data)')

# พล็อตเส้นการถดถอย (Regression Line)
# สร้างจุด X สำหรับวาดเส้นให้เรียบเนียน
X_line = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
y_line = model.predict(X_line)
plt.plot(X_line, y_line, color='red', linewidth=2, label='เส้นการถดถอย (Regression Line)')

# พล็อตเส้น Residuals (ความคลาดเคลื่อน)
# วาดเส้นแนวตั้งจากจุดข้อมูลจริงไปยังเส้นการถดถอย
for i in range(len(X)):
    plt.vlines(X[i], y_pred[i], y[i], color='gray', linestyle='dashed', alpha=0.5)

# แสดงคำอธิบายกราฟ
plt.title('Simple Linear Regression: ชั่วโมงการทบทวนบทเรียน vs คะแนนสอบปลายภาค', fontsize=14)
plt.xlabel('จำนวนชั่วโมงทบทวนบทเรียนต่อสัปดาห์ (ชั่วโมง)', fontsize=12)
plt.ylabel('คะแนนสอบปลายภาค', fontsize=12)
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

ผลลัพธ์:

```
สมการการถดถอยที่ได้: y_hat = 34.02 + 2.41X
```



alt text

การตีความผลลัพธ์จาก Code: เมื่อรันโปรแกรมข้างต้น สมมติว่าผลลัพธ์ที่แสดงทางหน้าจอคือ: `สมการการถดถอยที่ได้: y_hat = 32.15 + 2.85X`

นักศึกษาสามารถตีความค่าทางสถิติเพื่อนำไปใช้ในการวิเคราะห์พฤติกรรมการเรียนได้ดังนี้: 1. **Intercept ($\hat{\beta}_0$) = 32.15:** หากนักศึกษาไม่ได้ใช้เวลาในการทบทวนบทเรียนเลย (0 ชั่วโมงต่อสัปดาห์) คาดการณ์ว่านักศึกษาจะได้คะแนนสอบปลายภาคประมาณ 32.15 คะแนน (อาจมาจากความรู้พื้นฐาน หรือการตั้งใจเรียนในห้องเรียน) 2. **Slope ($\hat{\beta}_1$) = 2.85:** ทุกๆ 1 ชั่วโมงที่นักศึกษาใช้เวลาทบทวนบทเรียนเพิ่มขึ้นในหนึ่งสัปดาห์ จะส่งผลให้คะแนนสอบปลายภาคเพิ่มขึ้นโดยเฉลี่ย 2.85 คะแนน

กราฟที่ได้จะแสดงให้เห็นถึงจุดข้อมูล (Scatter) ที่มีแนวโน้มแปรผันตรง เส้นสีแดงคือเส้นที่ถูกคำนวณจากวิธีกำลังสองน้อยที่สุด (OLS) ซึ่งเป็นเส้นที่ทำให้ผลรวมของความยาวของเส้นประสีเทา (Residuals) ยกกำลังสองมีค่าน้อยที่สุดเท่าที่จะเป็นไปได้ (McKinney, 2022)

ข้อควรระวังและสมมติฐานเบื้องต้น

แม้ว่า Simple Linear Regression จะเป็นเครื่องมือที่มีประสิทธิภาพและตีความได้ง่าย แต่การนำไปใช้จำเป็นต้องตรวจสอบสมมติฐาน (Assumptions) ของข้อมูลด้วย หากข้อมูลละเมิดสมมติฐานเหล่านี้ ผลลัพธ์ที่ได้อาจขาดความน่าเชื่อถือ: 1. **Linearity:** ความสัมพันธ์ระหว่าง X และ Y ต้องเป็นเชิงเส้นตรง หากข้อมูลมีลักษณะเป็นเส้นโค้ง (เช่น Exponential หรือ Quadratic) การใช้สมการเส้นตรงจะไม่เหมาะสม 2. **Independence:** ข้อมูลแต่ละจุดต้องเป็นอิสระต่อกัน (ไม่มี Autocorrelation) 3. **Homoscedasticity:** ความแปรปรวนของ Residuals ต้องมีค่าคงที่ในทุกๆ ช่วงของ X (กราฟของ Residuals ไม่ควรมีลักษณะบานออกเหมือนรูปกรวย) 4. **Normality of Residuals:** ค่าความคลาดเคลื่อนควรมีการแจกแจงแบบปกติ (Normal Distribution)

การประเมินประสิทธิภาพของแบบจำลอง (Model Evaluation) เบื้องต้นมักจะใช้ค่า R^2 **(Coefficient of Determination)** ซึ่งมีค่าระหว่าง 0 ถึง 1 เพื่ออธิบายว่าตัวแปรอิสระ X สามารถอธิบายความผันผวนของตัวแปรตาม Y ได้กี่เปอร์เซ็นต์ ซึ่งรายละเอียดเชิงลึกของการประเมินแบบจำลองจะกล่าวถึงในส่วนถัดไปของบทเรียน

สรุปประเด็นสำคัญ

- **Simple Linear Regression** เป็นการสร้างสมการเส้นตรง $y = \beta_0 + \beta_1 x + \epsilon$ เพื่ออธิบายความสัมพันธ์และทำนายค่าตัวแปรตามทางปริมาณ 1 ตัวแปร จากตัวแปรอิสระ 1 ตัวแปร
- **Intercept (β_0)** คือค่าของ y เมื่อ $x = 0$ และ **Slope (β_1)** คืออัตราการเปลี่ยนแปลงของ y เมื่อ x เปลี่ยนไป 1 หน่วย
- **Ordinary Least Squares (OLS)** คืออัลกอริทึมทางคณิตศาสตร์ที่ใช้หาเส้นตรงที่เหมาะสมที่สุด โดยมีเป้าหมายในการทำให้ผลรวมของกำลังสองของความคลาดเคลื่อน (Sum of Squared Residuals) มีค่าน้อยที่สุด
- **Residual (e_i)** คือผลต่างระหว่างค่าข้อมูลจริงและค่าที่แบบจำลองทำนายได้ ($y_i - \hat{y}_i$) การแสดงผลด้วยกราฟเส้นประแนวตั้งช่วยให้เห็นภาพความคลาดเคลื่อนได้อย่างชัดเจน

คำถามท้าย Section

1. จากตัวอย่างสมการยอดขาย E-commerce $Y = 50 + 3.5X$ (หน่วยเป็นหมื่นบาท) หากบริษัทต้องการเพิ่มยอดขายให้ได้ 100 หมื่นบาท (1 ล้านบาท) บริษัทควรจัดสรรงบประมาณโฆษณา (X) อย่างน้อยเท่าใด? จงแสดงวิธีคำนวณ
2. เพราะเหตุใดในกระบวนการหาเส้นตรงที่เหมาะสมที่สุดของ OLS จึงต้องนำค่าความคลาดเคลื่อน (Residuals) มา “ยกกำลังสอง” ก่อนนำมารวมกัน? หากใช้เพียงค่าสัมบูรณ์ (Absolute value) จะส่งผลต่อกระบวนการคำนวณทางคณิตศาสตร์อย่างไร?
3. พิจารณาบริบทของโรงพยาบาล หากต้องการสร้างแบบจำลอง Simple Linear Regression เพื่อทำนาย “ระยะเวลาที่ผู้ป่วยต้องพักฟื้นในโรงพยาบาล (วัน)” นักศึกษาจะเลือกตัวแปรอิสระ (X) ใดมาใช้ในการวิเคราะห์ พร้อมอธิบายเหตุผลประกอบ

Multiple Linear Regression

การวิเคราะห์การถดถอยเชิงเส้นพหุคูณ (Multiple Linear Regression) เป็นการขยายขอบเขตจาก Simple Linear Regression ที่มีตัวแปรอิสระ (Independent Variable) เพียงตัวเดียว ให้สามารถรองรับตัวแปรอิสระหลายตัวได้ ในความเป็นจริง ข้อมูลที่พบในงานวิทยาการข้อมูลมักมีความซับซ้อนและผลลัพธ์ (Dependent Variable) มักได้รับอิทธิพลจากปัจจัยหลายประการพร้อมกัน ตัวอย่างเช่น ราคาของบ้านไม่ได้ขึ้นอยู่กับพื้นที่ใช้สอยเพียงอย่างเดียว แต่ยังขึ้นอยู่กับจำนวนห้องนอน ระยะทางจากสิ่งอำนวยความสะดวก และอายุของบ้าน เป็นต้น (James et al., 2023)

สมการทางคณิตศาสตร์ของ Multiple Linear Regression สามารถเขียนได้ดังนี้:

$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n + \epsilon$

โดยที่: * y คือ ค่าของตัวแปรตาม (Dependent Variable) หรือค่าที่ต้องการทำนาย * $x_1, x_2, \ldots, x_n$ คือ ค่าของตัวแปรอิสระ (Independent Variables) หรือ Features จำนวน n ตัว * β_0 คือ ค่าจุดตัดแกน y (y-intercept) ซึ่งหมายถึงค่าคาดหวังของ y เมื่อตัวแปรอิสระทุกตัวมีค่าเป็นศูนย์ * $\beta_1, \beta_2, \ldots, \beta_n$ คือ สัมประสิทธิ์การถดถอย (Regression Coefficients) ของตัวแปรอิสระแต่ละตัว * ϵ คือ ค่าความคลาดเคลื่อน (Error Term) ที่ไม่สามารถอธิบายได้ด้วยแบบจำลอง

การตีความสัมประสิทธิ์ (Interpreting Coefficients)

ในการวิเคราะห์ Multiple Linear Regression การตีความค่าสัมประสิทธิ์ β_i จะมีความซับซ้อนกว่าแบบจำลองตัวแปรเดียว ค่า β_i หมายถึง **การเปลี่ยนแปลงของค่าคาดหวังของ y เมื่อตัวแปร x_i เปลี่ยนแปลงไป 1 หน่วย โดยกำหนดให้ตัวแปรอิสระอื่นๆ ทั้งหมดมีค่าคงที่ (Holding all other variables constant)**

พิจารณาตัวอย่างการทำนายราคาคอนโดมิเนียมบริเวณรอบมหาวิทยาลัยขอนแก่น โดยมีสมการที่ได้จากการฝึกฝนแบบจำลอง (Training) ดังนี้:

$\text{Price} = 500,000 + 55,000(\text{Area}) - 15,000(\text{Distance})$

กำหนดให้: * `Price` คือ ราคาคอนโดมิเนียม (บาท) * `Area` คือ พื้นที่ห้อง (ตารางเมตร) * `Distance` คือ ระยะทางจากประตูมหาวิทยาลัย (กิโลเมตร)

จากการศึกษาสมการดังกล่าว สามารถตีความได้ดังนี้: 1. **$\beta_1 = 55,000$**: หากระยะทางจากมหาวิทยาลัยเท่ากัน คอนโดมิเนียมที่มีพื้นที่เพิ่มขึ้น 1 ตารางเมตร จะมีราคาเฉลี่ยเพิ่มขึ้น 55,000 บาท 2. **$\beta_2 = -15,000$**: หากพิจารณาคอนโดมิเนียมที่มีขนาดพื้นที่เท่ากัน เมื่อระยะทางห่างจากมหาวิทยาลัยเพิ่มขึ้น 1 กิโลเมตร ราคาเฉลี่ยของคอนโดมิเนียมจะลดลง 15,000 บาท 3. **$\beta_0 = 500,000$**: ในทางคณิตศาสตร์คือราคาของคอนโดมิเนียมขนาด 0 ตารางเมตรและอยู่ห่างจากมหาวิทยาลัย 0 กิโลเมตร ซึ่งในบริบทความเป็นจริงอาจไม่มีความหมายเชิงกายภาพ แต่เป็นค่าฐานที่ช่วยให้สมการมีความสมบูรณ์

สัมประสิทธิ์แบบดิบและแบบมาตรฐาน (Raw vs. Standardized Coefficients)

ปัญหาหนึ่งที่นักวิทยาศาสตร์ข้อมูลมักพบในการใช้ Multiple Linear Regression คือการเปรียบเทียบ “ความสำคัญ” (Feature Importance) ของตัวแปรอิสระแต่ละตัว หากพิจารณาจากสมการในหัวข้อ 8.3.1 อาจเกิดความเข้าใจผิดว่า `Area` (55,000) มีความสำคัญต่อราคามากกว่า `Distance` (-15,000) อย่างมาก แต่ในความเป็นจริง ตัวแปรทั้งสองมีหน่วยวัดและสเกลที่แตกต่างกันโดยสิ้นเชิง (ตารางเมตร vs กิโลเมตร)

เพื่อแก้ไขปัญหานี้ จึงมีการใช้ **Standardized Coefficients** เข้ามาช่วย (McKinney, 2022)

1. Raw Coefficients (สัมประสิทธิ์แบบดิบ): คือค่า β ที่ได้จากการคำนวณโดยตรงจากข้อมูลดิบ มีหน่วยตามตัวแปรนั้นๆ มีประโยชน์ในการนำไปใช้ทำนายค่าหรืออธิบายผลกระทบในหน่วยจริงที่ผู้ใช้งานธุรกิจเข้าใจได้ง่าย

2. Standardized Coefficients (สัมประสิทธิ์แบบมาตรฐาน): คือค่าสัมประสิทธิ์ที่ได้จากการปรับสเกลข้อมูล (Feature Scaling) ของทั้งตัวแปรอิสระ (X) และตัวแปรตาม (y) ให้เป็นคะแนนมาตรฐาน (Z-score) ก่อนนำไปสร้างแบบจำลอง โดยมีค่าเฉลี่ยเท่ากับ 0 และส่วนเบี่ยงเบนมาตรฐานเท่ากับ 1

$$z = \frac{x - \mu}{\sigma}$$

เมื่อใช้ข้อมูลที่ผ่านการทำ Standardize สมการจะไม่มีค่าจุดตัดแกน ($\beta_0 = 0$) และค่าสัมประสิทธิ์ที่ได้จะไม่มีหน่วย (Unitless) ทำให้สามารถเปรียบเทียบขนาด (Magnitude) ระหว่างตัวแปรได้โดยตรง ตัวแปรใดมีค่าสัมบูรณ์ (Absolute value) ของ Standardized Coefficient สูงกว่า ย่อมส่งผลต่อตัวแปรตามมากกว่า

คุณลักษณะ	Raw Coefficients	Standardized Coefficients
หน่วยวัด	มีหน่วยตามตัวแปรต้นทาง	ไม่มีหน่วย (Unitless)
การเปรียบเทียบความสำคัญ	ไม่สามารถเปรียบเทียบข้ามตัวแปรได้	เปรียบเทียบความสำคัญระหว่างตัวแปรได้โดยตรง
การนำไปใช้งาน	ใช้ทำนายผลลัพธ์ในบริบทจริง (เช่น คำนวณยอดขาย)	ใช้เพื่อหาว่า Feature ใดมีอิทธิพลสูงสุดในแบบจำลอง

ปัญหาภาวะร่วมเชิงเส้นพหุ (Multicollinearity)

ภาวะร่วมเชิงเส้นพหุ (Multicollinearity) คือสถานการณ์ที่ตัวแปรอิสระตั้งแต่สองตัวขึ้นไปในแบบจำลอง Multiple Linear Regression มีความสัมพันธ์เชิงเส้นระหว่างกันเองในระดับที่สูงมาก (Highly correlated) (James et al., 2023)

พิจารณาตัวอย่างระบบทำนายระยะเวลาการรักษาตัวของผู้ป่วยในโรงพยาบาล (วัน) โดยมีตัวแปรอิสระคือ น้ำหนัก (กิโลกรัม) และ ค่าดัชนีมวลกาย (BMI) เนื่องจากค่า BMI คำนวณมาจากน้ำหนักและส่วนสูง ตัวแปรทั้งสองจึงมีความสัมพันธ์กันอย่างหลีกเลี่ยงไม่ได้

ผลกระทบของ Multicollinearity: 1. **สัมประสิทธิ์ขาดความเสถียร (Unstable Coefficients):** การเปลี่ยนแปลงข้อมูลเพียงเล็กน้อยอาจส่งผลให้ค่า β เปลี่ยนแปลงไปอย่างมาก หรือแม้กระทั่งเปลี่ยนเครื่องหมายจากบวกเป็นลบ 2. **การตีความผิดพลาด:** เงื่อนไข “Holding all other variables constant” ไม่สามารถเป็นจริงได้ในทางปฏิบัติ ตัวอย่างเช่น ไม่สามารถเพิ่มน้ำหนักผู้ป่วยได้โดยที่ค่า BMI ยังคงที่ 3. **ค่าความคลาดเคลื่อนมาตรฐานสูงขึ้น (Inflated Standard Error):** ทำให้ค่า p-value สูงขึ้น ส่งผลให้ตัวแปรที่ควรจะมีนัยสำคัญทางสถิติ กลับกลายเป็นไม่มีนัยสำคัญ

Variance Inflation Factor (VIF)

เพื่อตรวจสอบว่าแบบจำลองมีปัญหา Multicollinearity หรือไม่ เครื่องมือที่ได้รับความนิยมและมีประสิทธิภาพที่สุดคือ **Variance Inflation Factor (VIF)**

ค่า VIF จะวัดระดับที่ความแปรปรวน (Variance) ของสัมประสิทธิ์การถดถอยเพิ่มขึ้นอันเนื่องมาจากตัวแปรอิสระนั้นมีความสัมพันธ์กับตัวแปรอิสระอื่นๆ สูตรการคำนวณ VIF สำหรับตัวแปร x_i คือ:

$$VIF_i = \frac{1}{1 - R_i^2}$$

โดยที่ R_i^2 คือค่า R-squared ที่ได้จากการนำตัวแปร x_i มาเป็นตัวแปรตาม และใช้ตัวแปรอิสระที่เหลือทั้งหมดในแบบจำลองมาเป็นตัวทำนาย

เกณฑ์การพิจารณาค่า VIF (Rule of Thumb): * **VIF = 1:** ตัวแปรอิสระไม่มีความสัมพันธ์กับตัวแปรอื่นๆ เลย (สมบูรณ์แบบ) * **1 < VIF < 5:** มีความสัมพันธ์ระดับปานกลาง (สามารถยอมรับได้) * **VIF ≥ 5:** มีปัญหา Multicollinearity ในระดับสูง ควรพิจารณาจัดการตัวแปรนี้ * **VIF ≥ 10:** มีปัญหา Multicollinearity ขั้นรุนแรง ต้องทำการแก้ไขทันที (เช่น การตัดตัวแปรทิ้ง หรือใช้เทคนิค Dimensionality Reduction อย่าง PCA)

หมายเหตุ: บางตำราอาจใช้เกณฑ์ VIF > 10 เป็นเกณฑ์ขั้นต่ำในการตัดตัวแปร ทั้งนี้ขึ้นอยู่กับบริบทของงานและความเข้มงวดของแบบจำลอง

การสร้างแบบจำลองด้วย Python

ในส่วนนี้ จะเป็นการสาธิตการสร้างแบบจำลอง Multiple Linear Regression โดยใช้ข้อมูลจำลองของธุรกิจ E-commerce ในประเทศไทย เพื่อทำนายยอดขาย (Sales) จากงบประมาณการยิงโฆษณาบน Facebook, TikTok และจำนวนโพสต์บน LINE OA พร้อมทั้งแสดงการคำนวณ Standardized Coefficients และ VIF

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from statsmodels.stats.outliers_influence import variance_inflation_factor
import statsmodels.api as sm

# 1. สร้างข้อมูลจำลอง (Synthetic Data) สำหรับธุรกิจ E-commerce
np.random.seed(42)
n_samples = 100

# จำลองงบโฆษณา (หน่วย: บาท)
fb_ads = np.random.normal(50000, 10000, n_samples)
tiktok_ads = np.random.normal(30000, 8000, n_samples)

# จงใจสร้าง Multicollinearity: ให้ Line Broadcast มีความสัมพันธ์สูงกับ FB Ads
line_broadcasts = fb_ads * 0.05 + np.random.normal(0, 100, n_samples)

# สร้างสมการยอดขาย + noise
# ยอดขายจริงขึ้นอยู่กับ FB และ TikTok แต่ไม่มีผลจาก Line โดยตรง
sales = 100000 + (2.5 * fb_ads) + (4.0 * tiktok_ads) + np.random.normal(0, 20000, n_samples)

# สร้าง DataFrame
df = pd.DataFrame({
    'FB_Ads_THB': fb_ads,
    'TikTok_Ads_THB': tiktok_ads,
    'Line_Broadcasts': line_broadcasts,
    'Sales_THB': sales
})

X = df[['FB_Ads_THB', 'TikTok_Ads_THB', 'Line_Broadcasts']]
y = df['Sales_THB']

# 2. การสร้างแบบจำลอง Multiple Linear Regression (Raw Coefficients)
model = LinearRegression()
model.fit(X, y)

print("--- Raw Coefficients ---")
print(f"Intercept: {model.intercept_:.2f}")
for feature, coef in zip(X.columns, model.coef_):
    print(f"{feature}: {coef:.4f}")

# 3. การหา Standardized Coefficients (เปรียบเทียบความสำคัญ)
scaler_X = StandardScaler()
scaler_y = StandardScaler()

# ปรับสเกลข้อมูลให้เป็น Z-score
X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y.values.reshape(-1, 1)).flatten()

model_std = LinearRegression()
model_std.fit(X_scaled, y_scaled)

print("--- Standardized Coefficients ---")
for feature, coef in zip(X.columns, model_std.coef_):
    print(f"{feature}: {coef:.4f}")

# 4. การตรวจสอบ Multicollinearity ด้วย VIF
print("--- Variance Inflation Factor (VIF) ---")
# Statsmodels ต้องการให้เติมค่าคงที่ (Constant) เข้าไปใน X เพื่อคำนวณ VIF ได้ถูกต้อง
X_vif = sm.add_constant(X)

vif_data = pd.DataFrame()
vif_data["Feature"] = X_vif.columns
vif_data["VIF"] = [variance_inflation_factor(X_vif.values, i) for i in range(len(X_vif.columns))]

# ตัดแถวที่เป็นค่าคงที่ (const) ออก เพื่อดูเฉพาะ VIF ของตัวแปรอิสระ
print(vif_data[vif_data['Feature'] != 'const'].to_string(index=False))

--- Raw Coefficients ---
Intercept: 122351.36
FB_Ads_THB: 1.8763
TikTok_Ads_THB: 3.9044
Line_Broadcasts: 5.3801

--- Standardized Coefficients ---
FB_Ads_THB: 0.4530
TikTok_Ads_THB: 0.7920
Line_Broadcasts: 0.0696

--- Variance Inflation Factor (VIF) ---
        Feature       VIF
     FB_Ads_THB 20.905768
 TikTok_Ads_THB  1.019085
Line_Broadcasts 20.900310
```

ผลลัพธ์ที่คาดหวังและการตีความ: จากการรันโค้ดข้างต้น จะพบว่า `FB_Ads_THB` และ `Line_Broadcasts` จะมีค่า VIF ที่สูงมาก (อาจเกิน 5 หรือ 10) เนื่องจากในขั้นตอนการสร้างข้อมูลจำลอง ได้กำหนดให้ `Line_Broadcasts` เป็นผลคูณเชิงเส้นของ `FB_Ads_THB` การมีอยู่ของตัวแปรทั้งสองพร้อมกันทำให้โมเดลขาดความเสถียร ในฐานะนักวิทยาศาสตร์ข้อมูล แนวทางแก้ไขคือการพิจารณาลบตัวแปรใดตัวแปรหนึ่งออก (เช่น ลบ `Line_Broadcasts`) แล้วทำการฝึกฝนแบบจำลองใหม่อีกครั้ง

ในส่วนของ Standardized Coefficients จะช่วยให้เห็นว่าระหว่างโฆษณา Facebook และ TikTok ช่องทางใดมีอิทธิพล (Impact) ต่อความแปรปรวนของยอดขายมากกว่ากัน โดยไม่ต้องกังวลเรื่องหน่วยของตัวแปรที่แตกต่างกัน

สรุปประเด็นสำคัญ

- **Multiple Linear Regression:** เป็นการประยุกต์ใช้สมการเชิงเส้นเพื่อทำนายผลลัพธ์จากตัวแปรอิสระหลายตัว โดยอิงสมมติฐานหลักคือตัวแปรอิสระแต่ละตัวมีผลกระทบต่อตัวแปรตามอย่างเป็นอิสระต่อกัน (เมื่อควบคุมตัวแปรอื่นให้คงที่)
- **Raw vs Standardized Coefficients:** Raw Coefficients ใช้ในการทำนายค่าและอธิบายผลในหน่วยจริง ส่วน Standardized Coefficients ใช้เพื่อเปรียบเทียบความสำคัญ (Feature Importance) ระหว่างตัวแปรอิสระที่มีสเกลต่างกัน
- **Multicollinearity:** คือปัญหาที่ตัวแปรอิสระมีความสัมพันธ์กันเองสูงเกินไป ส่งผลให้ค่าสัมประสิทธิ์ของแบบจำลองไม่เสถียรและตีความได้ยาก

- **VIF (Variance Inflation Factor):** เป็นมาตรวัดสำหรับตรวจจับ Multicollinearity หากพบตัวแปรที่มีค่า VIF เกิน 5 หรือ 10 ควรดำเนินการแก้ไขก่อนนำแบบจำลองไปใช้งานจริง

คำถามท้าย Section

1. หากได้สมการทำนายเงินเดือนเริ่มต้นของบัณฑิตจบใหม่คือ Salary = 15000 + 2000(GPA) + 500(TOEIC_Score) จงอธิบายความหมายของตัวเลข 2000 ตามหลักการของ Multiple Linear Regression
2. ในกรณีที่ต้องการนำเสนอผู้บริหารว่า ปัจจัยใดส่งผลต่ออัตราการลาออกของพนักงานมากที่สุด ระหว่าง “เงินเดือน (บาท)” กับ “ระยะเวลาการเดินทาง (นาที)” นักศึกษาควรนำเสนอค่าสัมประสิทธิ์ประเภทใด (Raw หรือ Standardized) เพราะเหตุใด?
3. หากนักศึกษาพบว่าตัวแปรอิสระ 2 ตัวในแบบจำลองมีค่า VIF เท่ากับ 12.5 นักศึกษาจะเสนอวิธีการแก้ไขปัญหานี้ในกระบวนการ Data Preparation อย่างไรได้บ้าง?

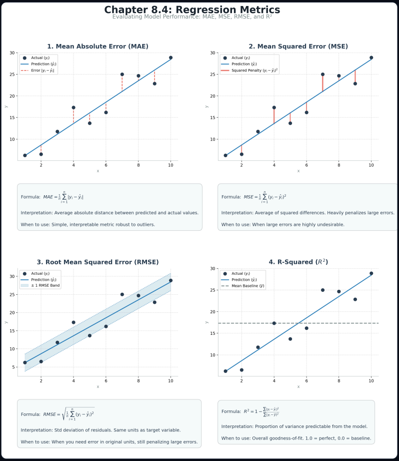
Evaluation Metrics

ในการพัฒนาโมเดลการเรียนรู้ของเครื่อง (Machine Learning) สำหรับปัญหาการวิเคราะห์การถดถอย (Regression) ขั้นตอนที่สำคัญอย่างยิ่งคือการประเมินประสิทธิภาพของโมเดล (Model Evaluation) เพื่อตรวจสอบว่าโมเดลที่สร้างขึ้นมีความแม่นยำเพียงใด และสามารถนำไปใช้งานจริงได้หรือไม่ การประเมินประสิทธิภาพในปัญหา Regression จะแตกต่างจากปัญหา Classification อย่างสิ้นเชิง เนื่องจากผลลัพธ์ที่ทำนาย (Predicted values) เป็นค่าต่อเนื่อง (Continuous values) ไม่ใช่หมวดหมู่ (Categorical values) ดังนั้น การวัดความถูกต้องจึงมุ่งเน้นไปที่การคำนวณ “ระยะห่าง” หรือ “ความคลาดเคลื่อน” (Error/Residual) ระหว่างค่าที่โมเดลทำนายได้ ($\hat{y}$) และค่าจริง (y) ที่ปรากฏในชุดข้อมูล (James et al., 2023)

พิจารณาสมการความคลาดเคลื่อนพื้นฐาน (Residual): $e_i = y_i - \hat{y}_i$

โดยที่: - y_i คือ ค่าจริง (Actual value) ของข้อมูลตัวที่ i - $\hat{y}_i$ คือ ค่าที่ทำนายได้ (Predicted value) ของข้อมูลตัวที่ i - n คือ จำนวนข้อมูลทั้งหมดในชุดข้อมูล

การนำค่าความคลาดเคลื่อน e_i ของทุกๆ จุดข้อมูลมารวมกันโดยตรงเพื่อหาค่าเฉลี่ยนั้นไม่สามารถทำได้ เนื่องจากความคลาดเคลื่อนมีทั้งค่าบวกและค่าลบ ซึ่งจะหักล้างกันเองจนอาจทำให้ผลรวมเข้าใกล้ศูนย์แม้ว่าโมเดลจะทำนายผิดพลาดอย่างมากก็ตาม ด้วยเหตุนี้ นักวิทยาศาสตร์ข้อมูลจึงพัฒนามาตรวัด (Evaluation Metrics) หลายรูปแบบเพื่อจัดการกับปัญหานี้ ซึ่งแต่ละมาตรวัดมีคุณสมบัติและการตีความที่แตกต่างกัน



Chapter 8.4: Regression Metrics

Evaluating Model Performance: MAE, MSE, RMSE, and R²

1. Mean Absolute Error (MAE)

Formula: $MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$

Interpretation: Average absolute distance between predicted and actual values.

When to use: Simple, interpretable metric robust to outliers.

2. Mean Squared Error (MSE)

Formula: $MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$

Interpretation: Average of squared differences. Heavily penalizes large errors.

When to use: When large errors are highly undesirable.

3. Root Mean Squared Error (RMSE)

Formula: $RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$

Interpretation: Std deviation of residuals. Same units as target variable.

When to use: When you need error in original units, still penalizing large errors.

4. R-Squared (R^2)

Formula: $R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$

Interpretation: Proportion of variance predictable from the model.

When to use: Overall goodness-of-fit. 1.0 = perfect, 0.0 = baseline.

Regression Metrics (MAE, MSE, RMSE, R²)

Mean Absolute Error (MAE)

Mean Absolute Error (MAE) คือ ค่าเฉลี่ยของความคลาดเคลื่อนสัมบูรณ์ เป็นการแก้ปัญหาการหักล้างกันของค่าบวกและค่าลบโดยการใส่ค่าสัมบูรณ์ (Absolute value) ให้กับความคลาดเคลื่อนของข้อมูลแต่ละจุด ก่อนที่จะนำมาหาค่าเฉลี่ย (Grus, 2019)

สมการทางคณิตศาสตร์:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

การตีความและคุณสมบัติ: - ค่า MAE มีหน่วยเดียวกับตัวแปรตาม (y) ทำให้สามารถตีความได้ง่ายและตรงไปตรงมา - MAE ให้ความสำคัญกับความคลาดเคลื่อนทุกจุดอย่างเท่าเทียมกัน (Linear penalty) หมายความว่าความคลาดเคลื่อนที่เพิ่มขึ้น 2 เท่า จะส่งผลให้ค่า MAE เพิ่มขึ้น 2 เท่าเช่นกัน - **ความทนทานต่อค่าผิดปกติ (Robustness to Outliers):** MAE มีความทนทานต่อค่าผิดปกติ (Outliers) ได้ดีกว่ามาตรวัดที่ใช้การยกกำลังสอง เนื่องจากไม่ได้ขยายขนาดของความคลาดเคลื่อนที่สูงผิดปกติ

ตัวอย่างในบริบทประเทศไทย: พิจารณาระบบทำนายระยะเวลาการจัดส่งพัสดุของบริษัท E-commerce ในประเทศไทย โดยมีหน่วยเป็น “วัน” หากโมเดลมีค่า MAE = 1.2 วัน สามารถตีความได้ว่า “โดยเฉลี่ยแล้ว โมเดลทำนายระยะเวลาการจัดส่งคลาดเคลื่อนไปจากความเป็นจริงประมาณ 1.2 วัน” ซึ่งเป็นตัวเลขที่ผู้บริหารหรือฝ่ายปฏิบัติการสามารถเข้าใจและนำไปประเมินความพึงพอใจของลูกค้าได้ทันที

Mean Squared Error (MSE)

Mean Squared Error (MSE) คือ ค่าเฉลี่ยของความคลาดเคลื่อนยกกำลังสอง เป็นมาตรวัดที่ได้รับความนิยมอย่างมากในทางสถิติและการเพิ่มประสิทธิภาพโมเดล (Optimization) เนื่องจากสมการยกกำลังสองมีคุณสมบัติที่สามารถหาอนุพันธ์ (Differentiable) ได้ง่าย ซึ่งเป็นประโยชน์ต่ออัลกอริทึมอย่าง Gradient Descent (VanderPlas, 2016)

สมการทางคณิตศาสตร์:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

การตีความและคุณสมบัติ: - การยกกำลังสองทำให้ความคลาดเคลื่อนที่เป็นลบกลายเป็นบวก และแก้ปัญหาการหักล้างกัน - **การลงโทษความคลาดเคลื่อนขนาดใหญ่ (Penalizing large errors):** จุดเด่นที่สำคัญที่สุดของ MSE คือการยกกำลังสองจะขยายขนาดของความคลาดเคลื่อนที่สูงให้มีผลกระทบต่อค่าเฉลี่ยรวมมากขึ้น (Quadratic penalty) โมเดลที่มี MSE ต่ำจึงเป็นโมเดลที่พยายามหลีกเลี่ยงการทำนายที่ผิดพลาดอย่างรุนแรง - หน่วยของ MSE จะเป็น “หน่วยกำลังสอง” ของตัวแปรตาม ซึ่งทำให้ยากต่อการตีความโดยตรง

ตัวอย่างในบริบทประเทศไทย: พิจารณาการทำนายความต้องการเตียงผู้ป่วยหนัก (ICU) ของโรงพยาบาลระดับศูนย์การแพทย์ (เช่น โรงพยาบาลศรีนครินทร์ มหาวิทยาลัยขอนแก่น) ในช่วงวิกฤตโรคระบาด การทำนายเตียงขาดไป 20 เตียงในวันเดียว ย่อมสร้างความเสียหายต่อชีวิตผู้ป่วยมากกว่าการทำนายเตียงขาดไป 4 เตียงเป็นเวลา 5 วัน (รวม 20 เตียงเท่ากัน) การใช้ MSE เป็นมาตรวัดจะช่วยบังคับให้โมเดลหลีกเลี่ยงความผิดพลาดขนาดใหญ่ (ขาด 20 เตียง) ได้ดีกว่าการใช้ MAE

Root Mean Squared Error (RMSE)

Root Mean Squared Error (RMSE) คือ รากที่สองของค่า MSE ถูกพัฒนาขึ้นเพื่อแก้ปัญหาเรื่อง “หน่วย” ของ MSE ที่ถูกยกกำลังสอง การถอดรากที่สองจะทำให้หน่วยของ RMSE กลับมาเป็นหน่วยเดียวกับตัวแปรตาม (y) ซึ่งช่วยให้ตีความได้ง่ายขึ้น ในขณะที่ยังคงคุณสมบัติการลงโทษความคลาดเคลื่อนขนาดใหญ่ไว้ (McKinney, 2022)

สมการทางคณิตศาสตร์:

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

การตีความและคุณสมบัติ: - มีหน่วยเดียวกับตัวแปรตาม ทำให้สื่อสารกับผู้ใช้งานระบบได้ง่าย - ยังคงมีความอ่อนไหวต่อค่าผิดปกติ (Sensitive to Outliers) เช่นเดียวกับ MSE - RMSE จะมีค่ามากกว่าหรือเท่ากับ MAE เสมอ ($RMSE \geq MAE$) ยิ่งความแตกต่างระหว่าง RMSE และ MAE มีมากเท่าใด แสดงว่าในชุดข้อมูลมีความคลาดเคลื่อนขนาดใหญ่ (Variance of individual errors) ปะปนอยู่มากเท่านั้น

ตัวอย่างในบริบทประเทศไทย: ในการสร้างโมเดลทำนายราคาคอนโดมิเนียมตามแนวรถไฟฟ้าในกรุงเทพมหานคร หากใช้ MSE หน่วยที่ได้จะเป็น “บาทตาราง” ซึ่งไม่มีความหมายในโลกความเป็นจริง แต่เมื่อคำนวณเป็น RMSE อาจได้ค่าเท่ากับ 250,000 บาท สามารถตีความได้ว่า “โดยเฉลี่ยแล้ว โมเดลทำนายราคาคอนโดมิเนียมคลาดเคลื่อนไปประมาณ 250,000 บาท โดยมีการให้น้ำหนักกับความผิดพลาดที่มีมูลค่าสูงๆ ด้วย”

Coefficient of Determination (R^2)

มาตรวัด MAE, MSE และ RMSE ที่กล่าวมาข้างต้น เป็นมาตรวัดที่ขึ้นอยู่กับ “สเกล” (Scale-dependent) ของข้อมูล หากเปลี่ยนหน่วยจากบาทเป็นล้านบาท ค่าของมาตรวัดเหล่านี้จะเปลี่ยนไป ทำให้ไม่สามารถเปรียบเทียบประสิทธิภาพข้ามชุดข้อมูลได้ **Coefficient of Determination** หรือ **R^2 (R-squared)** จึงถูกนำมาใช้เพื่อแก้ปัญหานี้ โดย R^2 เป็นมาตรวัดเชิงเปรียบเทียบ (Relative metric) ที่ไม่มีหน่วย (James et al., 2023)

R^2 วัดสัดส่วนของความแปรปรวน (Variance) ในตัวแปรตาม (y) ที่สามารถอธิบายได้ด้วยตัวแปรอิสระ (X) ผ่านโมเดลที่สร้างขึ้น

สมการทางคณิตศาสตร์:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

โดยที่: - SS_{res} (Sum of Squares of Residuals) คือ ผลรวมความคลาดเคลื่อนยกกำลังสองของโมเดล: $\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$ - SS_{tot} (Total Sum of Squares) คือ ผลรวมความแปรปรวนรวมของข้อมูล (เสมือนใช้ค่าเฉลี่ย $\bar{y}$ ทำนายทุกจุด): $\sum_{i=1}^{n}(y_i - \bar{y})^2$

การตีความและคุณสมบัติ: - ค่า R^2 โดยทั่วไปจะมีค่าอยู่ระหว่าง 0 ถึง 1 (สามารถติดลบได้หากโมเดลทำนายได้แย่กว่าการใช้ค่าเฉลี่ยทายผล) - $R^2 = 1$: โมเดลสามารถอธิบายความแปรปรวนของข้อมูลได้ 100% (ทำนายถูกต้องสมบูรณ์แบบ) - $R^2 = 0$: โมเดลไม่สามารถอธิบายข้อมูลได้ดีไปกว่าการใช้ค่าเฉลี่ยของ y มาทำนาย - **ข้อจำกัดที่สำคัญ:** ค่า R^2 จะไม่ลดลงและมีแนวโน้มที่จะเพิ่มขึ้นเสมอ เมื่อมีการเพิ่มตัวแปรอิสระ (Features) เข้าไปในโมเดล แม้ว่าตัวแปรนั้นจะไม่มีความสัมพันธ์ใดๆ กับตัวแปรตามเลยก็ตาม ซึ่งอาจนำไปสู่ปัญหา Overfitting

ตัวอย่างในบริบทประเทศไทย: พิจารณาโมเดลทำนายเกรดเฉลี่ยสะสม (GPAX) ของนักศึกษามหาวิทยาลัยขอนแก่น โดยใช้คะแนนสอบเข้า (TCAS) เป็นตัวแปรอิสระ หากโมเดลมีค่า $R^2 = 0.65$ สามารถตีความทางสถิติได้ว่า “ร้อยละ 65 ของความแตกต่างในเกรดเฉลี่ยสะสมของนักศึกษา สามารถอธิบายได้ด้วยคะแนนสอบเข้า TCAS ผ่านโมเดลนี้” ส่วนอีก 35% ที่เหลือเกิดจากปัจจัยอื่นๆ ที่ไม่ได้นำมาคำนวณ เช่น ความขยัน สภาพแวดล้อม หรือกิจกรรมเสริมหลักสูตร

Adjusted R^2

เนื่องจากข้อจำกัดของ R^2 ที่มีค่าเพิ่มขึ้นเสมอเมื่อเพิ่มจำนวนตัวแปรอิสระ **Adjusted R^2** จึงถูกพัฒนาขึ้นเพื่อปรับแก้ปัญหานี้ โดยตัวสมการจะมีการเพิ่มบทลงโทษ (Penalty) เมื่อมีการเพิ่มตัวแปรอิสระที่ไม่ได้ช่วยให้โมเดลมีประสิทธิภาพดีขึ้นอย่างมีนัยสำคัญ (McKinney, 2022)

สมการทางคณิตศาสตร์:

$$Adjusted\ R^2 = 1 - \left[\frac{(1 - R^2)(n - 1)}{n - p - 1}\right]$$

โดยที่: - n คือ จำนวนจุดข้อมูล (Sample size) - p คือ จำนวนตัวแปรอิสระ (Number of predictors/features)

การตีความและคุณสมบัติ: - Adjusted R^2 จะมีค่าน้อยกว่าหรือเท่ากับ R^2 เสมอ - หากเพิ่มตัวแปรอิสระที่มีประโยชน์จริง ค่า Adjusted R^2 จะเพิ่มขึ้น - หากเพิ่มตัวแปรอิสระที่ไม่มีประโยชน์ ค่า Adjusted R^2 จะลดลง (เนื่องจากตัวหาร $n - p - 1$ มีค่าน้อยลง ทำให้ก้อนที่นำไปลบออกจาก 1 มีค่ามากขึ้น) - เหมาะอย่างยิ่งสำหรับการเปรียบเทียบโมเดล Linear Regression หลายๆ โมเดลที่มีจำนวนตัวแปรอิสระไม่เท่ากัน (Feature Selection)

ตัวอย่างในบริบทประเทศไทย: สมมติว่าธนาคารแห่งหนึ่งต้องการสร้างโมเดลประเมินวงเงินสินเชื่อบ้าน โมเดลเริ่มต้นใช้ตัวแปร 2 ตัว คือ “รายได้ประจำ” และ “ประวัติการค้างชำระ” ได้ค่า $R^2 = 0.80$ และ Adjusted $R^2 = 0.79$ ต่อมา ทีมพัฒนาได้เพิ่มตัวแปร “สีรถยนต์ที่ผู้กู้ขับ” เข้าไปในโมเดล ค่า R^2 อาจขยับขึ้นเล็กน้อยเป็น 0.801 (เพราะคณิตศาสตร์บังคับให้โมเดลจับรูปแบบแบบสุ่มได้บ้าง) แต่ค่า Adjusted R^2 จะลดลงเหลือตัวอย่างเช่น 0.78 แสดงให้เห็นว่าตัวแปร “สีรถยนต์” ไม่ได้ให้ข้อมูลที่เป็นประโยชน์ต่อการทำนาย และไม่ควรนำมารวมในโมเดล

การเปรียบเทียบและเลือกใช้ Metrics

การเลือกใช้มาตรวัดที่เหมาะสมขึ้นอยู่กับวัตถุประสงค์ทางธุรกิจและลักษณะของชุดข้อมูล ตารางด้านล่างสรุปความแตกต่างเพื่อช่วยในการตัดสินใจ

มาตรวัด (Metric)	การตีความ (Interpretation)	ความอ่อนไหวต่อ Outlier	ข้อควรพิจารณาในการใช้งาน (Best Use Case)
MAE	ค่าเฉลี่ยของความคลาดเคลื่อน (หน่วยเดิม)	ต่ำ (Robust)	เมื่อต้องการรายงานผลให้ผู้ใช้งานทั่วไปเข้าใจง่าย และไม่ต้องการให้ Outlier ดึงประสิทธิภาพโมเดลลงมากเกินไป
MSE	ค่าเฉลี่ยความคลาดเคลื่อนกำลังสอง	สูงมาก	เมื่อต้องการลงโทษความผิดพลาดขนาดใหญ่ (เช่น การแพทย์, การเงิน) และใช้ในกระบวนการ Optimization ของอัลกอริทึม
RMSE	รากที่สองของ MSE (หน่วยเดิม)	สูง	เมื่อต้องการคุณสมบัติแบบ MSE แต่ต้องการหน่วยที่ตีความได้ง่ายขึ้น
R^2	สัดส่วนความแปรปรวนที่โมเดลอธิบายได้	ปานกลาง	เมื่อต้องการวัดประสิทธิภาพรวมของโมเดลโดยไม่ผูกกับหน่วยของข้อมูล เพื่อเปรียบเทียบข้ามบริบท
Adjusted R^2	R^2 ที่ปรับแก้ตามจำนวนตัวแปร	ปานกลาง	เมื่อต้องการเปรียบเทียบโมเดลที่มีจำนวน Feature ต่างกัน ป้องกันปัญหา Overfitting จากสุ่มเพิ่ม Feature

การเขียนโปรแกรมด้วย Python (Scikit-Learn)

ในภาษา Python ไลบรารี `scikit-learn` ได้เตรียมฟังก์ชันสำหรับการคำนวณ Evaluation Metrics เหล่านี้ไว้อย่างครบถ้วนในโมดูล `sklearn.metrics` ตัวอย่างโค้ดด้านล่างแสดงการจำลองชุดข้อมูลการทำนายค่าไฟฟ้าของครัวเรือน และประเมินผลด้วยมาตรวัดทั้ง 5 ชนิด

```python
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# 1. จำลองข้อมูล (Simulated Data)
# สมมติว่าเป็นชุดข้อมูลค่าไฟฟ้าของครัวเรือน 10 หลัง
# y_true คือ ค่าไฟฟ้าจริงที่บิลเรียกเก็บ (หน่วย: บาท)
y_true = np.array([1200, 1500, 1800, 1100, 2500, 1400, 900, 3200, 1600, 1350])

# y_pred คือ ค่าไฟฟ้าที่โมเดลทำนายได้จากตัวแปรต่างๆ (เช่น จำนวนคน, ขนาดแอร์)
y_pred = np.array([1150, 1600, 1750, 1150, 2100, 1450, 950, 2800, 1650, 1300])

# จำนวนข้อมูล (n) และ จำนวนตัวแปรอิสระที่ใช้สมมติว่าใช้ 3 ตัว (p)
n = len(y_true)
p = 3

# 2. คำนวณ Evaluation Metrics
# 2.1 MAE
mae = mean_absolute_error(y_true, y_pred)

# 2.2 MSE
mse = mean_squared_error(y_true, y_pred)

# 2.3 RMSE (สามารถใช้ mean_squared_error พร้อมกำหนด squared=False ใน sklearn เวอร์ชันเก่า
# หรือถอดรากที่สองด้วย np.sqrt ในเวอร์ชันใหม่)
rmse = np.sqrt(mse)

# 2.4 R-squared
r2 = r2_score(y_true, y_pred)

# 2.5 Adjusted R-squared (คำนวณตามสูตรคณิตศาสตร์ เนื่องจากไม่มีฟังก์ชันสำเร็จรูปใน sklearn)
adj_r2 = 1 - ((1 - r2) * (n - 1) / (n - p - 1))

# 3. แสดงผลลัพธ์
print("ผลการประเมินประสิทธิภาพโมเดลทำนายค่าไฟฟ้า:")
print("-" * 40)
print(f"MAE          : {mae:.2f} บาท")
print(f"MSE          : {mse:.2f} บาท^2")
print(f"RMSE         : {rmse:.2f} บาท")
print(f"R-squared    : {r2:.4f}")
print(f"Adjusted R^2 : {adj_r2:.4f}")

# การตีความเบื้องต้น:
# - โมเดลทายค่าไฟคลาดเคลื่อนเฉลี่ยประมาณ 125 บาท (MAE)
# - ค่า RMSE (186.41) สูงกว่า MAE แสดงว่ามีความคลาดเคลื่อนสูงในบางจุด (เช่น จุดที่ค่าไฟจริง 3200 แต่ทาย 2800)
# - โมเดลสามารถอธิบายความแปรปรวนของค่าไฟฟ้าได้ถึง 92.09% (R-squared = 0.9209) ถือว่ามีประสิทธิภาพสูง
```

ผลลัพธ์:

```
ผลการประเมินประสิทธิภาพโมเดลทำนายค่าไฟฟ้า:
----------------------------------------
MAE          : 125.00 บาท
MSE          : 34750.00 บาท^2
RMSE         : 186.41 บาท
R-squared    : 0.9209
Adjusted R^2 : 0.8813
```

จากการศึกษา Evaluation Metrics ทั้งหมดในส่วนนี้ นักศึกษาจะเห็นได้ว่าไม่มีมาตรวัดใดที่ “ดีที่สุด” ในทุกสถานการณ์ การเป็นนักวิทยาศาสตร์ข้อมูลที่ดี ไม่ใช่เพียงแค่การเขียนโค้ดเพื่อเรียกดูค่าตัวเลขเหล่านี้ได้ แต่คือความสามารถในการ **“เลือก”** มาตรวัดที่สอดคล้องกับบริบทของปัญหา และสามารถ **“ตีความ”** ตัวเลขทางสถิติให้เป็นภาษาที่ผู้มีส่วนได้ส่วนเสีย (Stakeholders) ทางธุรกิจสามารถนำไปใช้ประกอบการตัดสินใจได้อย่างมีประสิทธิภาพ

สรุปประเด็นสำคัญ

- **MAE และ RMSE** มีหน่วยเดียวกับเป้าหมายการทำนาย ทำให้ตีความได้ง่ายในโลกความเป็นจริง แต่ RMSE จะลงโทษความผิดพลาดขนาดใหญ่รุนแรงกว่า MAE
- **MSE** เป็นรากฐานสำคัญในกระบวนการเพิ่มประสิทธิภาพโมเดล (Optimization) เนื่องจากสมการทางคณิตศาสตร์สามารถหาอนุพันธ์ได้ง่าย
- R^2 ใช้สำหรับประเมินว่าโมเดลสามารถอธิบายความแปรปรวนของข้อมูลได้ดีเพียงใด โดยเป็นค่าสัมพัทธ์ที่ไม่มีหน่วย ทำให้เปรียบเทียบข้ามชุดข้อมูลได้
- **Adjusted** R^2 เป็นเครื่องมือสำคัญในการทำ Feature Selection เนื่องจากมีกลไกลงโทษการเพิ่มตัวแปรอิสระที่ไม่มีประโยชน์ ซึ่งช่วยป้องกันปัญหา Overfitting

คำถามท้าย Section

1. หากนักศึกษาได้รับมอบหมายให้สร้างโมเดลทำนายอายุการใช้งานของเครื่องจักรในโรงงานอุตสาหกรรม โดยมีเงื่อนไขว่า “หากเครื่องจักรเสียก่อนเวลาที่ทำนายไว้ จะเกิดความเสียหายต่อสายการผลิตอย่างมหาศาล” นักศึกษาควรเลือกใช้ MAE หรือ RMSE เป็นมาตรวัดหลักในการเลือกโมเดลที่ดีที่สุด? จงอธิบายเหตุผล
2. ในการสร้างโมเดล Linear Regression โมเดล A มีตัวแปรอิสระ 5 ตัว ได้ค่า R^2 = 0.85 และโมเดล B มีตัวแปรอิสระ 20 ตัว ได้ค่า R^2 = 0.86 นักศึกษาคิดว่าโมเดล B เป็นโมเดลที่ดีกว่าโมเดล A อย่างแท้จริงหรือไม่? ควรใช้มาตรวัดใดในการตรวจสอบสมมติฐานนี้?

Regression Diagnostics

การสร้างโมเดล Linear Regression ไม่ได้สิ้นสุดเพียงแค่การคำนวณหาค่าสัมประสิทธิ์ (Coefficients) หรือการวัดค่าประสิทธิภาพด้วย R^2 และ Root Mean Squared Error (RMSE) เท่านั้น ในทางสถิติและวิทยาการข้อมูล โมเดล Linear Regression ตั้งอยู่บนสมมติฐาน (Assumptions) ทางคณิตศาสตร์หลายประการ หากชุดข้อมูลที่นำมาใช้ละเมิดสมมติฐานเหล่านี้ โมเดลที่ได้อาจให้ผลการทำนายที่เอนเอียง (Biased) หรือให้ค่าความเชื่อมั่น (Confidence Intervals) ที่ผิดพลาด ซึ่งนำไปสู่การตัดสินใจทางธุรกิจที่คลาดเคลื่อนได้ (James et al., 2023)

กระบวนการตรวจสอบความถูกต้องของโมเดลเทียบกับสมมติฐานพื้นฐานนี้ เรียกว่า **Regression Diagnostics** ซึ่งอาศัยการวิเคราะห์ค่าความคลาดเคลื่อน หรือ **Residuals** (e) เป็นหลัก โดย Residual คือผลต่างระหว่างค่าจริง (y_i) และค่าที่โมเดลทำนายได้ ($\hat{y}_i$) ดังสมการ:

$e_i = y_i - \hat{y}_i$

สมมติฐานพื้นฐานของ Linear Regression (LINE Assumptions)

การตรวจสอบโมเดล Linear Regression มักพิจารณาจากสมมติฐาน 4 ประการหลัก ซึ่งสามารถจำด้วยตัวย่อ **LINE** ดังนี้:

1. L - Linearity (ความสัมพันธ์เชิงเส้น) สมมติฐานนี้ระบุว่าความสัมพันธ์ระหว่างตัวแปรอิสระ (X) และตัวแปรตาม (Y) ต้องเป็นเส้นตรง หากความสัมพันธ์ที่แท้จริงเป็นเส้นโค้ง (เช่น Exponential หรือ Quadratic) การใช้ Linear Regression จะทำให้โมเดลเกิด Underfitting อย่างรุนแรง * **บริบทตัวอย่าง:** การทำนายยอดขายของธุรกิจ e-commerce ในประเทศไทย จากงบประมาณโฆษณาบนโซเชียลมีเดีย โดยปกติแล้วยอดขายอาจเพิ่มขึ้นอย่างรวดเร็วในช่วงแรก แต่เมื่อถึงจุดหนึ่งจะเกิดภาวะอิ่มตัว (Diminishing Returns) ซึ่งเป็นความสัมพันธ์แบบไม่เป็นเส้นเส้นตรง (Non-linear)

2. I - Independence (ความเป็นอิสระของข้อมูล) ค่า Residuals ของแต่ละจุดข้อมูลต้องเป็นอิสระต่อกัน (Independent) นั่นคือ ค่าความคลาดเคลื่อนของข้อมูลจุดที่ i ต้องไม่ส่งผลหรือมีความสัมพันธ์กับค่าความคลาดเคลื่อนของข้อมูลจุดที่ $i + 1$ * **บริบทตัวอย่าง:** การทำนายจำนวนผู้ป่วยโรคไข้เลือดออกรายวันของโรงพยาบาลในจังหวัดขอนแก่น ข้อมูลลักษณะนี้เป็นข้อมูลอนุกรมเวลา (Time Series) ซึ่งจำนวนผู้ป่วยในวันนี้ย่อมมีความสัมพันธ์กับจำนวนผู้ป่วยในวันพรุ่งนี้ (Autocorrelation) ทำให้ละเมิดสมมติฐานข้อนี้

3. N - Normality of Residuals (การแจกแจงแบบปกติของค่าความคลาดเคลื่อน) สมมติฐานนี้กำหนดให้ค่า Residuals มีการแจกแจงแบบปกติ (Normal Distribution) โดยมีค่าเฉลี่ยเท่ากับศูนย์ แม้ว่าการละเมิดสมมติฐานข้อนี้อาจไม่ได้ทำให้ค่าสัมประสิทธิ์ของโมเดลผิดเพี้ยนไป แต่จะทำให้การทดสอบสมมติฐานทางสถิติ (เช่น p-value) และการสร้างช่วงความเชื่อมั่น (Confidence Interval) ขาดความน่าเชื่อถือ (McKinney, 2022)

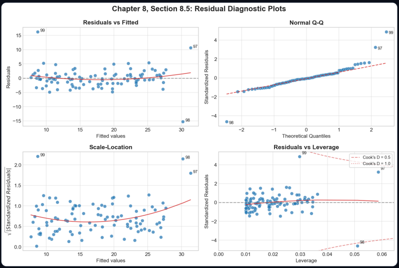
4. E - Equal Variance หรือ Homoscedasticity (ความแปรปรวนคงที่) ความแปรปรวน (Variance) ของค่า Residuals ต้องมีค่าคงที่ในทุกๆ ระดับของตัวแปรอิสระ (X) หากความแปรปรวนมีขนาดเปลี่ยนไป (เช่น กว้างขึ้นเมื่อค่า X มากขึ้น) จะเรียกว่าเกิดสภาวะ **Heteroscedasticity** * **บริบทตัวอย่าง:** การทำนายราคาคอนโดมิเนียมในกรุงเทพมหานครจากพื้นที่ใช้สอย (ตารางเมตร) คอนโดขนาดเล็กมักมีช่วงราคาที่เกาะกลุ่มกัน (ความแปรปรวนต่ำ) แต่คอนโดขนาดใหญ่หรือ Penthouse อาจมีราคาที่แตกต่างกันอย่างมหาศาลขึ้นอยู่กับทำเลและการตกแต่ง (ความแปรปรวนสูง) ทำให้เกิด Heteroscedasticity

เครื่องมือวิเคราะห์ (Diagnostic Plots)

ในการตรวจสอบสมมติฐาน LINE นักวิทยาศาสตร์ข้อมูลนิยมใช้การสร้างภาพนิทัศน์ (Data Visualization) จากค่า Residuals แทนการดูเพียงตัวเลขสถิติ เครื่องมือที่สำคัญมีดังนี้

1. Residuals vs Fitted Plot กราฟนี้ใช้ตรวจสอบ **Linearity** และ **Homoscedasticity** โดยแกน X คือค่าที่โมเดลทำนายได้ (Fitted values, $\hat{y}$) และแกน Y คือค่า Residuals (e) * **ลักษณะที่ดี:** จุดข้อมูลควรกระจายตัวแบบสุ่ม (Random Scatter) รอบเส้นศูนย์ (y = 0) โดยไม่มีรูปแบบ (Pattern) ชัดเจน และความกว้างของการกระจายตัวควรสม่ำเสมอตลอดแนวแกน X

```
[Diagram: รูปแบบของ Residuals vs Fitted Plot]

แบบที่ 1: ปกติ (Homoscedasticity)   แบบที่ 2: ผิดปกติ (Heteroscedasticity)   แบบที่ 3: ผิดปกติ (Non-linear)
  e |   *   *    *                  e |        *                          e |   *       *

    | *   *   *    *                  |     *   *                           |     *   *
  0 |---*---*---*----               0 |---*--*---*-----                   0 |------*----

    |   *   *                         |         *                           |
    +--------------- Fitted           +--------------- Fitted               +--------------- Fitted
    (กระจายตัวสม่ำเสมอ)                  (รูปกรวย / Cone shape)                  (รูปตัว U หรือโค้ง)
```



Residual Analysis Plots

2. Normal Q-Q Plot (Quantile-Quantile Plot) ใช้สำหรับตรวจสอบ **Normality** โดยเปรียบเทียบการกระจายตัวของ Residuals ที่ได้จากโมเดล (Sample Quantiles) กับการกระจายตัวแบบปกติในอุดมคติ (Theoretical Quantiles) * **ลักษณะที่ดี:** จุดข้อมูลควรเรียงตัวทับหรือใกล้เคียงกับเส้นทแยงมุม 45 องศา หากส่วนหัวหรือส่วนหางของกราฟเบี่ยงเบนออกจากเส้นตรงอย่างมาก แสดงว่าข้อมูลมี Outliers หรือมีการแจกแจงแบบหางหนา (Heavy-tailed)

วงจร Problem → Diagnose → Fix

ในการทำงานจริง เมื่อพบว่าโมเดลละเมิดสมมติฐานข้อใดข้อหนึ่ง นักวิทยาศาสตร์ข้อมูลจะต้องดำเนินการแก้ไขเพื่อปรับปรุงโมเดล กระบวนการนี้ทำงานเป็นวงจร (Cycle) ดังตารางต่อไปนี้

สมมติฐานที่ละเมิด (Problem)	วิธีการตรวจสอบ (Diagnose)	แนวทางการแก้ไข (Fix)
Non-linearity (ความสัมพันธ์ไม่เป็นเส้นตรง)	กราฟ Residuals vs Fitted มีรูปแบบเป็นเส้นโค้ง (เช่น ตัว U)	1. เพิ่ม Polynomial Features (X^2, X^3) 2. แปลงรูปตัวแปร X เช่น log (X) หรือ $\sqrt{X}$
Heteroscedasticity (ความแปรปรวนไม่คงที่)	กราฟ Residuals vs Fitted มีลักษณะบานออกเป็นรูปกรวย (Funnel shape)	1. แปลงรูปตัวแปรตาม Y เช่น log (Y) 2. ใช้โมเดล Weighted Least Squares (WLS)
Non-normality (Residuals ไม่แจกแจงแบบปกติ)	กราฟ Q-Q Plot เบี่ยงเบนออกจากเส้นทแยงมุม 45 องศาอย่างชัดเจน	1. ตรวจสอบและจัดการ Outliers 2. ใช้ Box-Cox Transformation
Autocorrelation (ข้อมูลไม่อิสระต่อกัน)	พล็อตกราฟ Residuals ตามลำดับเวลา (Time order) แล้วพบรูปแบบเป็นคลื่น	1. เพิ่มตัวแปร Lag (เช่น Y_{t-1}) 2. เปลี่ยนไปใช้โมเดล Time Series เช่น ARIMA

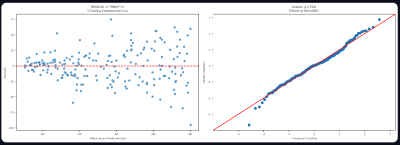
การประยุกต์ใช้ด้วย Python

ในภาษา Python ไลบรารี `scikit-learn` มีประสิทธิภาพสูงในการสร้างโมเดล Machine Learning แต่มีข้อจำกัดด้านเครื่องมือทางสถิติ ดังนั้นในการทำ Regression Diagnostics จึงนิยมใช้ไลบรารี `statsmodels` ซึ่งให้ผลลัพธ์เชิงสถิติที่ละเอียดกว่า (VanderPlas, 2016)

พิจารณาสถานการณ์จำลอง: การสร้างโมเดลทำนายค่าใช้จ่ายในการบำรุงรักษาระบบเซิร์ฟเวอร์ของมหาวิทยาลัย (หน่วย: หมื่นบาท) จากปริมาณการใช้งานแบนด์วิดท์ (TB)

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import scipy.stats as stats

# 1. สร้างข้อมูลจำลอง (จำลองสถานการณ์ที่มี Heteroscedasticity)
np.random.seed(42)
bandwidth_tb = np.random.uniform(10, 100, 200)

# สร้างค่าใช้จ่าย โดยให้ความแปรปรวน (Noise) เพิ่มขึ้นตามปริมาณแบนด์วิดท์
# ละเมิดสมมติฐาน Homoscedasticity อย่างตั้งใจ
noise = np.random.normal(0, bandwidth_tb * 0.5)
maintenance_cost = 50 + (2.5 * bandwidth_tb) + noise

df = pd.DataFrame({'Bandwidth': bandwidth_tb, 'Cost': maintenance_cost})

# 2. สร้างโมเดล Linear Regression ด้วย statsmodels
X = sm.add_constant(df['Bandwidth']) # statsmodels ต้องเติมค่าคงที่ (Intercept) เอง
Y = df['Cost']
model = sm.OLS(Y, X).fit()

# ดึงค่า Fitted values และ Residuals ออกมาจากโมเดล
fitted_vals = model.predict(X)
residuals = model.resid

# 3. สร้าง Diagnostic Plots (Diagnose)
fig, ax = plt.subplots(1, 2, figsize=(14, 5))

# Plot 1: Residuals vs Fitted Plot
sns.scatterplot(x=fitted_vals, y=residuals, ax=ax[0], alpha=0.7)
ax[0].axhline(0, color='red', linestyle='--')
ax[0].set_xlabel('Fitted Values (Predicted Cost)')
ax[0].set_ylabel('Residuals')
ax[0].set_title('Residuals vs Fitted Plot(Checking Homoscedasticity)')

# Plot 2: Normal Q-Q Plot
sm.qqplot(residuals, line='45', fit=True, ax=ax[1])
ax[1].set_title('Normal Q-Q Plot(Checking Normality)')

plt.tight_layout()
plt.show()
```

ผลลัพธ์:



Regression Diagnostics
กราฟซ้าย (Residuals vs Fitted Plot): จุดข้อมูลกระจายตัวบานออกทางด้านขวาคล้ายรูปกรวย บ่งบอกปัญหา Heteroscedasticity — ความแปรปรวนเพิ่มขึ้นเมื่อค่า Fitted Values สูงขึ้น
กราฟขวา (Normal Q-Q Plot): จุดข้อมูลส่วนใหญ่อยู่ใกล้เส้นทแยงมุม 45° แต่มีการเบี่ยงเบนที่หางทั้งสองด้าน

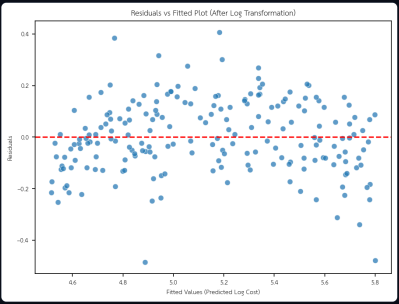
จากการรันโค้ดด้านบน หากพิจารณาที่กราฟ **Residuals vs Fitted Plot** จะพบว่าการกระจายตัวของ Residuals มีลักษณะบานออกทางด้านขวาคล้ายรูปกรวย (Cone shape) เมื่อค่า Fitted Values สูงขึ้น ค่า Residuals ก็แกว่งกว้างขึ้นตามไปด้วย นี่คือสัญญาณที่ชัดเจนของปัญหา **Heteroscedasticity**

การแก้ไข (Fix): ตามวงจร Problem → Diagnose → Fix วิธีหนึ่งในการแก้ปัญหาความแปรปรวนไม่คงที่ คือการแปลงรูปตัวแปรตาม (Log Transformation) เพื่อบีบอัดความแปรปรวนของค่าที่มีขนาดใหญ่

```python
# 4. การแก้ไข (Fix) ด้วย Log Transformation
df['Log_Cost'] = np.log(df['Cost'])

# สร้างโมเดลใหม่ด้วย Log(Y)
Y_log = df['Log_Cost']
model_fixed = sm.OLS(Y_log, X).fit()

fitted_vals_fixed = model_fixed.predict(X)
```

```
residuals_fixed = model_fixed.resid

# สร้าง Plot เพื่อตรวจสอบอีกครั้ง
plt.figure(figsize=(7, 5))
sns.scatterplot(x=fitted_vals_fixed, y=residuals_fixed, alpha=0.7)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Fitted Values (Predicted Log Cost)')
plt.ylabel('Residuals')
plt.title('Residuals vs Fitted Plot (After Log Transformation)')
plt.show()
```

ผลลัพธ์:



Diagnostics After Fix
กราฟ Residuals vs Fitted (หลัง Log Transformation): จุดข้อมูลกระจายตัวรอบเส้นศูนย์อย่างสม่ำเสมอมากขึ้น ไม่บานออกเป็นรูปกรวยเหมือนก่อนแก้ไข แสดงว่าปัญหา Heteroscedasticity ลดลงอย่างมีนัยสำคัญ

จากการแก้ไขด้วย Log Transformation จะพบว่ากราฟ Residuals vs Fitted มีการกระจายตัวของข้อมูลรอบเส้นศูนย์ที่สม่ำเสมอมากขึ้น (Random Scatter) ซึ่งบ่งชี้ว่าโมเดลใหม่มีความสอดคล้องกับสมมติฐาน Homoscedasticity มากขึ้น ทำให้ค่าสถิติและการทำนายช่วงความเชื่อมั่นมีความน่าเชื่อถือทางสถิติ

การทำ Regression Diagnostics จึงไม่ใช่เพียงขั้นตอนทางเลือก แต่เป็นกระบวนการบังคับในวิถีของ Data Science เพื่อยืนยันว่าโมเดลที่ส่งมอบนั้น “ถูกต้องตามหลักการ” และ “ใช้งานได้จริง” อย่างปลอดภัย

สรุปประเด็นสำคัญ

- **Regression Diagnostics** คือกระบวนการตรวจสอบว่าโมเดล Linear Regression ละเมิดสมมติฐานทางสถิติหรือไม่ โดยอาศัยการวิเคราะห์ค่าความคลาดเคลื่อน (Residuals)
- สมมติฐานหลัก 4 ประการจำได้ด้วยคำว่า **LINE**: Linearity (เชิงเส้น), Independence (อิสระ), Normality (แจกแจงแบบปกติ), และ Equal Variance/Homoscedasticity (ความแปรปรวนคงที่)
- **Residuals vs Fitted Plot** ใช้ตรวจสอบความสัมพันธ์เชิงเส้นและความแปรปรวนคงที่ จุดข้อมูลที่ดีควรกระจายตัวแบบสุ่มรอบเส้นศูนย์
- **Q-Q Plot** ใช้ตรวจสอบว่า Residuals มีการแจกแจงแบบปกติหรือไม่ โดยจุดข้อมูลควรทาบไปกับเส้นทแยงมุม 45 องศา
- กระบวนการปรับปรุงโมเดลทำผ่านวงจร **Problem** → **Diagnose** → **Fix** เช่น การแก้ปัญหา Heteroscedasticity ด้วยการทำ Log Transformation

คำถามท้าย Section

1. หากนักศึกษาใช้โมเดล Linear Regression ทำนายราคาบ้านจัดสรรในประเทศไทย แล้วพบว่ากราฟ Residuals vs Fitted มีลักษณะเป็นรูปตัวยู (U-shape) นักศึกษาจะอธิบายสาเหตุของปัญหานี้อย่างไร และมีแนวทางแก้ไขเบื้องต้นอย่างไร?
2. สมมติฐาน Independence (ความเป็นอิสระของข้อมูล) มักจะถูกละเมิดเมื่อวิเคราะห์ข้อมูลประเภทใด จงยกตัวอย่างข้อมูลในบริบทของธุรกิจค้าปลีกมา 1 ตัวอย่าง

การปรับปรุง Model

กระบวนการสร้างแบบจำลองการถดถอย (Regression Model) มิได้สิ้นสุดเพียงการนำอัลกอริทึมมาเรียนรู้ชุดข้อมูลและได้ผลลัพธ์ออกมา ในทางปฏิบัติ โมเดลเริ่มต้น (Baseline Model) มักเผชิญกับปัญหาหลายประการ เช่น ปัญหาการเรียนรู้เกิน (Overfitting) ปัญหาความสัมพันธ์ที่ไม่เป็นเส้นตรง (Non-linear Relationship) หรือปัญหาตัวแปรอิสระที่ไม่มีความสำคัญต่อการทำนาย ส่งผลให้ประสิทธิภาพของโมเดลเมื่อนำไปใช้งานกับข้อมูลจริง (Unseen Data) ลดต่ำลง การปรับปรุงโมเดลจึงเป็นขั้นตอนสำคัญที่นักวิทยาศาสตร์ข้อมูลต้องดำเนินการเพื่อให้ได้โมเดลที่มีความแม่นยำและมีเสถียรภาพ (Robustness) (James et al., 2023)

การคัดเลือกคุณลักษณะ (Feature Selection)

ในยุคของ Big Data ชุดข้อมูลมักประกอบด้วยคุณลักษณะ (Features) จำนวนมหาศาล อย่างไรก็ตาม การนำคุณลักษณะทั้งหมดมาใช้สร้างโมเดลไม่ได้หมายความว่าจะได้โมเดลที่ดีเสมอไป การมีคุณลักษณะที่มากเกินไปอาจนำไปสู่ปัญหา “คำสาปของมิติ” (Curse of Dimensionality) ซึ่งทำให้โมเดลมีความซับซ้อนเกินความจำเป็น ใช้เวลาในการประมวลผลสูง และเสี่ยงต่อการเกิด Overfitting นอกจากนี้ คุณลักษณะบางประการอาจเป็นเพียงสัญญาณรบกวน (Noise) หรือมีความสัมพันธ์กันเองสูง (Multicollinearity) (McKinney, 2022)

พิจารณาตัวอย่างระบบทำนายความสามารถในการชำระหนี้ของลูกค้าธนาคารพาณิชย์ในประเทศไทย ข้อมูลอาจประกอบด้วย รายได้ต่อเดือน, ภาระหนี้สินเดิม, ประวัติการชำระเงินล่าช้า, และ “สีรถยนต์ที่ขับ” จะเห็นได้ว่า “สีรถยนต์” เป็นคุณลักษณะที่ไม่มีความสัมพันธ์เชิงตรรกะกับความสามารถในการชำระหนี้ การตัดคุณลักษณะนี้ออกจะช่วยให้โมเดลมีประสิทธิภาพดีขึ้น

วิธีการคัดเลือกคุณลักษณะ (Feature Selection Methods) โดยทั่วไปแบ่งออกเป็น 3 กลุ่มหลัก: 1. **Filter Methods:** การใช้สถิติเพื่อประเมินความสัมพันธ์ระหว่างแต่ละ Feature กับ Target Variable เช่น การหาค่า Correlation (Pearson’s r) หรือ f-statistic วิธีนี้คำนวณได้รวดเร็วและไม่ขึ้นอยู่กับโมเดล 2. **Wrapper Methods:** การทดลองสร้างโมเดลด้วยชุดของ Features ที่แตกต่างกัน แล้วประเมินประสิทธิภาพ เช่น Recursive Feature Elimination (RFE) วิธีนี้ให้ผลลัพธ์ที่แม่นยำแต่ใช้ทรัพยากรคำนวณสูง 3. **Embedded Methods:** อัลกอริทึมทำการคัดเลือก Features ไปพร้อมกับการฝึกสอนโมเดล (Training) เช่น Lasso Regression หรือ Decision Trees

```
import pandas as pd
import numpy as np
from sklearn.datasets import make_regression
from sklearn.feature_selection import SelectKBest, f_regression

# จำลองข้อมูลนักศึกษามหาวิทยาลัยขอนแก่น (100 คน, 5 Features)
# สมมติให้ y คือ เกรดเฉลี่ย (GPAX)
# X ประกอบด้วย: [ชั่วโมงอ่านหนังสือ, รายได้ครอบครัว, ระยะทางจากหอพัก, ตัวแปรรบกวน1, ตัวแปรรบกวน2]
X, y = make_regression(n_samples=100, n_features=5, n_informative=2, noise=0.5, random_state=42)

# กำหนดชื่อคอลัมน์เพื่อความชัดเจน
feature_names = ['Study_Hours', 'Family_Income', 'Distance_to_Uni', 'Noise_1', 'Noise_2']
df_X = pd.DataFrame(X, columns=feature_names)

# ใช้ Filter Method (SelectKBest ร่วมกับ f_regression) เลือก 2 Features ที่ดีที่สุด
selector = SelectKBest(score_func=f_regression, k=2)
X_selected = selector.fit_transform(df_X, y)

# แสดงผลคะแนนความสำคัญของแต่ละ Feature
scores = selector.scores_
for name, score in zip(feature_names, scores):
    print(f"Feature: {name:15} | F-Score: {score:.2f}")

# ดึงชื่อ Feature ที่ถูกเลือก
selected_features = np.array(feature_names)[selector.get_support()]
print(f"Features ที่ถูกเลือก: {selected_features}")
```

ผลลัพธ์:

```
Feature: Study_Hours     | F-Score: 0.25
Feature: Family_Income   | F-Score: 76.27
Feature: Distance_to_Uni | F-Score: 1.66
Feature: Noise_1         | F-Score: 65.06
Feature: Noise_2         | F-Score: 0.30

Features ที่ถูกเลือก: ['Family_Income' 'Noise_1']
```

คำถามชวนคิด: หากชุดข้อมูลทางการแพทย์มีผู้ป่วยเพียง 50 คน แต่มีข้อมูลยีน (Genes) ถึง 10,000 ยีน นักศึกษาคิดว่าควรใช้วิธีการคัดเลือกคุณลักษณะแบบใดเพื่อหลีกเลี่ยงปัญหา Overfitting โดยที่ทรัพยากรคอมพิวเตอร์มีจำกัด?

การแปลงข้อมูล (Data Transformation)

โมเดล Linear Regression มีข้อสมมติฐานสำคัญประการหนึ่งคือ ตัวแปรอิสระ (X) และตัวแปรตาม (y) ต้องมีความสัมพันธ์กันเชิงเส้นตรง (Linearity) ทว่าในบริบทของโลกความเป็นจริง ข้อมูลจำนวนมากมิได้มีความสัมพันธ์เช่นนั้น การแปลงข้อมูล (Transformation) จึงเป็นเทคนิคทางคณิตศาสตร์ที่ใช้ปรับเปลี่ยนรูปทรงการกระจายตัวของข้อมูล หรือสร้างคุณลักษณะใหม่ เพื่อให้โมเดลสามารถเรียนรู้รูปแบบที่ซ่อนอยู่ได้อย่างมีประสิทธิภาพ (VanderPlas, 2016)

การแปลงด้วยลอการิทึม (Log Transform)

ข้อมูลทางธุรกิจและเศรษฐศาสตร์มักมีการกระจายตัวแบบเบ้ขวา (Right-skewed Distribution) หรือมีค่าผิดปกติ (Outliers) ที่มีค่าสูงมาก ตัวอย่างเช่น ข้อมูลราคาคอนโดมิเนียมตามแนวรถไฟฟ้าในกรุงเทพมหานคร คอนโดส่วนใหญ่อาจมีราคา 2-5 ล้านบาท แต่มีคอนโดระดับ Super Luxury จำนวนน้อยที่มีราคาสูงถึง 100 ล้านบาท การนำข้อมูลดิบนี้ไปสร้างโมเดลจะทำให้โมเดลถูกดึง (Pull) โดยค่า Outliers เหล่านี้ ส่งผลให้การทำนายกลุ่มคนส่วนใหญ่คลาดเคลื่อน

การใช้ Log Transform ($y' = \log(y)$) จะช่วยบีบอัดช่วงของข้อมูลที่มีค่าสูงๆ ให้แคบลง ส่งผลให้การกระจายตัวเข้าใกล้การแจกแจงแบบปกติ (Normal Distribution) มากขึ้น และลดอิทธิพลของ Outliers ในภาษา Python มักใช้ `numpy.log1p(x)` ซึ่งคำนวณ $\log(1 + x)$ เพื่อป้องกันปัญหาการหาค่า Log ของศูนย์

```
import numpy as np
import matplotlib.pyplot as plt

# จำลองข้อมูลยอดสั่งซื้อบนแพลตฟอร์ม E-commerce (มีลักษณะเบ้ขวา)
np.random.seed(42)
purchase_amounts = np.random.exponential(scale=1000, size=1000)

# ทำการแปลงด้วย Log Transform
log_purchase_amounts = np.log1p(purchase_amounts)

# แสดงค่าสถิติเชิงพรรณนาเปรียบเทียบ
print(f"ข้อมูลดิบ - ค่าเฉลี่ย: {np.mean(purchase_amounts):.2f}, ค่าสูงสุด: {np.max(purchase_amounts):.2f}")
print(f"ข้อมูล Log - ค่าเฉลี่ย: {np.mean(log_purchase_amounts):.2f}, ค่าสูงสุด: {np.max(log_purchase_amounts):.2f}")
```

ผลลัพธ์:

```
ข้อมูลดิบ - ค่าเฉลี่ย: 972.51, ค่าสูงสุด: 8172.45
ข้อมูล Log - ค่าเฉลี่ย: 6.30, ค่าสูงสุด: 9.01
```

การสร้างคุณลักษณะแบบพหุนาม (Polynomial Features)

เมื่อความสัมพันธ์ระหว่าง X และ y เป็นเส้นโค้ง การใช้สมการเส้นตรง ($y = \beta_0 + \beta_1 X$) จะนำไปสู่ปัญหา Underfitting พิจารณาตัวอย่างการใช้พลังงานไฟฟ้าของอาคารเรียนรวมในมหาวิทยาลัยขอนแก่น เทียบกับอุณหภูมิภายนอก เมื่ออุณหภูมิต่ำ (อากาศเย็น) การใช้ไฟฟ้าอาจสูงจากเครื่องทำน้ำอุ่น หรือเมื่ออุณหภูมิสูง (อากาศร้อน) การใช้ไฟฟ้าก็จะสูงจากการเปิดเครื่องปรับอากาศ ความสัมพันธ์นี้มีลักษณะเป็นรูปตัวยู (U-shape)

ในกรณีนี้ เราสามารถแปลงข้อมูลด้วย Polynomial Features โดยเพิ่มพจน์ยกกำลังของตัวแปรเดิมเข้าไป เช่น การใช้พหุนามดีกรี 2 สมการจะเปลี่ยนเป็น $y = \beta_0 + \beta_1 X + \beta_2 X^2$ แม้สมการจะมีพจน์ X^2 แต่ในมุมมองของ Machine Learning โมเดลนี้ยังคงเป็น “Linear Regression” เนื่องจากสัมประสิทธิ์ (β) ยังคงเป็นเชิงเส้น

```
from sklearn.preprocessing import PolynomialFeatures
import numpy as np

# สมมติ X คือ อุณหภูมิ (องศาเซลเซียส)
X_temp = np.array([[20], [25], [30], [35], [40]])

# สร้าง Polynomial Features ดีกรี 2
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X_temp)

# X_poly จะประกอบด้วย [X, X^2]
print("ข้อมูลเดิม (X):", X_temp)
print("ข้อมูลหลังแปลง (X, X^2):", X_poly)
```

ผลลัพธ์:

```
ข้อมูลเดิม (X):
 [[20]
 [25]
 [30]
 [35]
 [40]]

ข้อมูลหลังแปลง (X, X^2):
 [[  20.  400.]
 [  25.  625.]
 [  30.  900.]
 [  35. 1225.]
 [  40. 1600.]]
```

คำถามชวนคิด: การเพิ่มระดับดีกรี (Degree) ของ Polynomial Features เป็นระดับ 10 หรือ 20 จะส่งผลดีหรือผลเสียต่อโมเดลอย่างไร? จงอธิบายโดยอ้างอิงถึงแนวคิด Bias-Variance Tradeoff

---

Regularization: การลดความซับซ้อนของ Model

เมื่อเราพยายามเพิ่มความซับซ้อนให้กับโมเดล (เช่น การเพิ่มตัวแปรจำนวนมาก หรือการใช้ Polynomial Degree สูงๆ) เพื่อให้โมเดลทำนายข้อมูลฝึกสอน (Training Data) ได้แม่นยำที่สุด โมเดลมักจะจดจำแม้กระทั่ง Noise ในข้อมูล นำไปสู่ปัญหา **Overfitting** ซึ่งโมเดลจะมีค่า Variance สูง และไม่สามารถนำไปทำนายข้อมูลใหม่ได้

Regularization คือเทคนิคการเพิ่ม “ค่าปรับ” (Penalty Term) เข้าไปในฟังก์ชันสูญเสีย (Loss Function) ของโมเดล เพื่อบังคับให้ค่าสัมประสิทธิ์ (β) มีค่าน้อยลง หรือกล่าวอีกนัยหนึ่งคือ เป็นการลงโทษโมเดลที่มีความซับซ้อนมากเกินไป (Grus, 2019)

สมการ Loss Function พื้นฐานของ Linear Regression (Mean Squared Error ย่อว่า MSE) คือ:

$$Loss = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

เมื่อเพิ่ม Regularization สมการจะกลายเป็น: $Loss = MSE + Penalty$

เทคนิค Regularization ที่ได้รับความนิยมใน Regression มี 2 ประเภทหลัก ได้แก่ Ridge และ Lasso

Ridge Regression (L2 Regularization)

Ridge Regression เพิ่มค่าปรับโดยคำนวณจาก **ผลรวมของกำลังสองของสัมประสิทธิ์** (Sum of squared coefficients)

$$Loss_{Ridge} = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda\sum_{j=1}^{p}\beta_j^2$$

- λ (หรือ `alpha` ใน scikit-learn) คือ Hyperparameter ที่ควบคุมระดับการลงโทษ
    - หาก $\lambda = 0$ โมเดลจะเทียบเท่ากับ Linear Regression ปกติ
    - หาก λ มีค่าสูงมาก ค่าสัมประสิทธิ์ β จะถูกบีบให้เข้าใกล้ศูนย์
- **จุดเด่น:** Ridge มีประสิทธิภาพสูงในการจัดการกับปัญหา Multicollinearity (เมื่อตัวแปรอิสระมีความสัมพันธ์กันเองสูง) โดย Ridge จะกระจายค่าน้ำหนัก (Weights) ให้กับตัวแปรที่สัมพันธ์กันอย่างเท่าเทียมกัน และจะไม่ตัดตัวแปรใดทิ้งไปอย่างสมบูรณ์ (สัมประสิทธิ์เข้าใกล้ศูนย์ แต่ไม่เท่ากับศูนย์)

Lasso Regression (L1 Regularization)

Lasso (Least Absolute Shrinkage and Selection Operator) เพิ่มค่าปรับโดยคำนวณจาก **ผลรวมของค่าสัมบูรณ์ของสัมประสิทธิ์** (Sum of absolute coefficients)

$$Loss_{Lasso} = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda\sum_{j=1}^{p}|\beta_j|$$

- **จุดเด่น:** ลักษณะทางคณิตศาสตร์ของ L1 Norm ส่งผลให้เมื่อค่า λ เพิ่มขึ้น สัมประสิทธิ์ของตัวแปรที่มีความสำคัญน้อยจะถูกบีบให้ **เท่ากับศูนย์อย่างสมบูรณ์**
- คุณสมบัตินี้ทำให้ Lasso ทำหน้าที่เสมือนเป็น **Embedded Feature Selection** โดยอัตโนมัติ เหมาะสำหรับชุดข้อมูลที่มี Features จำนวนมาก แต่มีเพียงไม่กี่ Features ที่ส่งผลต่อ Target Variable จริงๆ

ตารางเปรียบเทียบ Ridge และ Lasso

คุณลักษณะ	Ridge Regression (L2)	Lasso Regression (L1)
Penalty Term	$\lambda\sum\beta^2$	$
การลดรูปค่าสัมประสิทธิ์	บีบให้เข้าใกล้ 0 แต่ไม่เท่ากับ 0	สามารถบีบให้เท่ากับ 0 ได้
Feature Selection	ไม่สามารถทำได้	ทำได้โดยอัตโนมัติ
ข้อได้เปรียบหลัก	จัดการ Multicollinearity ได้ดีมาก	สร้างโมเดลที่ตีความง่าย (Sparse Model)
บริบทการใช้งาน	เมื่อเชื่อว่าทุกตัวแปรมีความสำคัญ	เมื่อเชื่อว่ามีเพียงบางตัวแปรที่สำคัญ

ตัวอย่างการใช้งานในบริบทสาธารณสุข

พิจารณาการทำนายค่าใช้จ่ายในการรักษาพยาบาลของผู้ป่วยในโรงพยาบาลศูนย์ขอนแก่น โดยมีตัวแปรอิสระ 100 ตัวแปร (เช่น อายุ, น้ำหนัก, ความดันเลือด, ประวัติพันธุกรรม 50 โรค, และข้อมูลที่ไม่เกี่ยวข้องเช่น รายการอาหารที่ชอบ ฯลฯ)

```
import numpy as np
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# 1. จำลองข้อมูลผู้ป่วย 1000 คน, 100 Features (มีเพียง 10 Features ที่มีผลจริง)
np.random.seed(42)
X = np.random.randn(1000, 100)
# สร้างสัมประสิทธิ์ 100 ตัว (10 ตัวมีค่า, 90 ตัวเป็นศูนย์)
true_coef = np.zeros(100)
true_coef[:10] = np.array([10, 20, 15, -10, 5, 25, -5, 30, 12, -15])
y = np.dot(X, true_coef) + np.random.normal(0, 5, 1000) # เพิ่ม Noise

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. สร้างโมเดลทั้ง 3 แบบ
lr = LinearRegression()
ridge = Ridge(alpha=10.0) # alpha คือค่า lambda ในสมการ
lasso = Lasso(alpha=1.0)

# 3. Train Models
```

```
lr.fit(X_train, y_train)
ridge.fit(X_train, y_train)
lasso.fit(X_train, y_train)

# 4. ประเมินผล (นับจำนวนสัมประสิทธิ์ที่เป็นศูนย์)
print(f"Linear Regression: จำนวนสัมประสิทธิ์ที่เป็น 0 = {np.sum(lr.coef_ == 0)}/100")
print(f"Ridge Regression : จำนวนสัมประสิทธิ์ที่เป็น 0 = {np.sum(ridge.coef_ == 0)}/100")
print(f"Lasso Regression : จำนวนสัมประสิทธิ์ที่เป็น 0 = {np.sum(lasso.coef_ == 0)}/100")

# 5. ประเมินความแม่นยำ (MSE บน Test Set)
print(f"MSE (Linear) : {mean_squared_error(y_test, lr.predict(X_test)):.2f}")
print(f"MSE (Ridge)  : {mean_squared_error(y_test, ridge.predict(X_test)):.2f}")
print(f"MSE (Lasso)  : {mean_squared_error(y_test, lasso.predict(X_test)):.2f}")
```

ผลลัพธ์:

```
Linear Regression: จำนวนสัมประสิทธิ์ที่เป็น 0 = 0/100
Ridge Regression : จำนวนสัมประสิทธิ์ที่เป็น 0 = 0/100
Lasso Regression : จำนวนสัมประสิทธิ์ที่เป็น 0 = 90/100

MSE (Linear) : 27.24
MSE (Ridge)  : 26.35
MSE (Lasso)  : 30.94
```

จากการศึกษาผลลัพธ์ของโค้ดข้างต้น พบว่า Lasso สามารถตรวจจับและปรับลดสัมประสิทธิ์ของตัวแปรที่ไม่เกี่ยวข้อง (90 ตัวแปร) ให้กลายเป็นศูนย์ได้ ส่งผลให้โมเดลที่ได้มีความเรียบง่าย (Sparse) และมีค่าความคลาดเคลื่อน (MSE) บนชุดข้อมูลทดสอบต่ำกว่า Linear Regression ปกติอย่างชัดเจน

คำถามชวนคิด: หากนักศึกษาพบว่าตัวแปรอิสระในชุดข้อมูลมีความสัมพันธ์กันเองสูงมาก (เช่น พื้นที่ใช้สอยของบ้าน กับ จำนวนห้องนอน) การเลือกใช้ Lasso อาจทำให้เกิดปัญหาใด? และเพราะเหตุใด Ridge จึงอาจเป็นตัวเลือกที่เหมาะสมกว่าในกรณีนี้?

สรุปประเด็นสำคัญ

- **การคัดเลือกคุณลักษณะ (Feature Selection):** เป็นกระบวนการลดมิติข้อมูลเพื่อขจัดสัญญาณรบกวน ลดความซับซ้อนของคำนวณ และป้องกันปัญหา Overfitting โดยสามารถใช้วิธีการทางสถิติ (Filter) หรือพึ่งพาอัลกอริทึม (Wrapper/Embedded)
- **การแปลงข้อมูล (Data Transformation):** Log Transform ใช้แก้ปัญหาข้อมูลเบ้ขวาและลดผลกระทบจาก Outliers ในขณะที่ Polynomial Features ใช้เพื่อจับรูปแบบความสัมพันธ์ที่ไม่เป็นเส้นตรงระหว่างตัวแปร
- **Regularization:** เป็นการเพิ่ม Penalty term เข้าไปใน Loss Function เพื่อลงโทษโมเดลที่มีความซับซ้อนเกินไป ช่วยแก้ปัญหา Overfitting
- **Ridge vs. Lasso:** Ridge (L2) จัดการกับ Multicollinearity ได้ดีโดยลดขนาดสัมประสิทธิ์แต่ไม่ตัดทิ้ง ส่วน Lasso (L1) สามารถบีบสัมประสิทธิ์ให้เป็นศูนย์ได้ จึงทำหน้าที่เสมือนเป็นเครื่องมือคัดเลือกคุณลักษณะ (Feature Selection) ในตัว

คำถามท้าย Section

1. สมมติว่านักศึกษากำลังสร้างโมเดลทำนายราคาประเมินที่ดินในจังหวัดขอนแก่น โดยมีข้อมูลพื้นที่ (ตารางวา) และราคาประเมิน เมื่อพล็อตข้อมูลพบว่ายิ่งพื้นที่ขนาดใหญ่ ราคาจะทวีคูณขึ้นเป็นเส้นโค้ง นักศึกษาจะใช้เทคนิคการปรับปรุงโมเดลใดเพื่อแก้ปัญหานี้ พร้อมอธิบายเหตุผล
2. จงอธิบายความแตกต่างเชิงคณิตศาสตร์ระหว่างค่า Penalty ของ Ridge และ Lasso และความแตกต่างนี้ส่งผลต่อค่าสัมประสิทธิ์ (Coefficients) อย่างไรเมื่อนำไปใช้งานจริง?
3. ในกระบวนการทำงานจริง นักวิทยาศาสตร์ข้อมูลควรทำ Data Transformation (เช่น Log Transform) ก่อน หรือหลังจากการแบ่งข้อมูลเป็น Training Set และ Test Set? เพราะเหตุใด?

Polynomial Regression: การจัดการความสัมพันธ์ที่ไม่เป็นเส้นตรง

ในหัวข้อก่อนหน้า ได้ศึกษา Linear Regression ซึ่งสร้างเส้นตรงเพื่ออธิบายความสัมพันธ์ระหว่างตัวแปร อย่างไรก็ตาม ในโลกแห่งความเป็นจริง ความสัมพันธ์ระหว่างตัวแปรหลายกรณี **ไม่ได้เป็นเส้นตรง** เช่น ราคาที่ดินที่เพิ่มขึ้นแบบทวีคูณตามขนาดพื้นที่ หรือยอดขายที่มีจุดอิ่มตัว (Saturation point) เมื่อปริมาณโฆษณาเพิ่มขึ้น

หลักการ Polynomial Regression

Polynomial Regression คือการขยาย Linear Regression ด้วยการ **สร้างตัวแปรใหม่** จากตัวแปรเดิมยกกำลัง ตัวอย่างเช่น หาก Features เดิมคือ x เราสามารถสร้าง Features ใหม่ $x^2, x^3, \ldots, x^d$ แล้วนำไป Fit กับ Linear Regression ปกติ

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \ldots + \beta_d x^d + \epsilon$$

แม้สมการจะมีกำลังสูง แต่โมเดลนี้ยังคงเป็น “Linear” ในมุมมองของ Parameters (β) ดังนั้นจึงยังใช้ scikit-learn `LinearRegression` ได้ เพียงแต่ต้องเตรียมข้อมูลด้วย `PolynomialFeatures` ก่อน

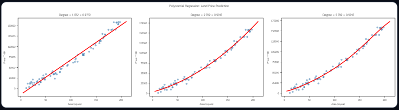
```
Linear:      y = b0 + b1*x                     (เส้นตรง)
Quadratic:   y = b0 + b1*x + b2*x^2            (พาราโบลา)
Cubic:       y = b0 + b1*x + b2*x^2 + b3*x^3   (เส้น S-curve)
```

ตัวอย่าง: ทำนายราคาที่ดินจังหวัดขอนแก่น

สมมติว่าราคาประเมินที่ดินในเขตอำเภอเมืองขอนแก่น มีความสัมพันธ์แบบโค้งกับขนาดพื้นที่ (ตารางวา) — ที่ดินขนาดเล็กราคาต่อตารางวาสูง แต่ยิ่งใหญ่ขึ้นราคาจะเพิ่มขึ้นแบบทวีคูณ

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# 1. สร้างข้อมูลจำลอง: ขนาดพื้นที่ vs ราคา (ความสัมพันธ์แบบ Quadratic)
np.random.seed(42)
X = np.random.uniform(10, 200, 80).reshape(-1, 1)  # ขนาด (ตร.วา)
y = 5000 + 300 * X.flatten() + 2.5 * X.flatten()**2 + np.random.normal(0, 5000, 80)  # ราคา (บาท)

# 2. เปรียบเทียบ degree = 1 vs 2 vs 5
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
X_plot = np.linspace(5, 210, 100).reshape(-1, 1)
```

```python
for ax, degree in zip(axes, [1, 2, 5]):
    # สร้าง Polynomial Features
    poly = PolynomialFeatures(degree=degree, include_bias=False)
    X_poly = poly.fit_transform(X)
    X_plot_poly = poly.transform(X_plot)

    # Fit โมเดล
    model = LinearRegression()
    model.fit(X_poly, y)
    y_plot = model.predict(X_plot_poly)
    r2 = r2_score(y, model.predict(X_poly))

    # พล็อต
    ax.scatter(X, y, alpha=0.5, s=15, color='steelblue')
    ax.plot(X_plot, y_plot, color='red', linewidth=2)
    ax.set_title(f'Degree = {degree} (R2 = {r2:.3f})')
    ax.set_xlabel('Area (sq.wa)')
    ax.set_ylabel('Price (THB)')

plt.suptitle('Polynomial Regression: Land Price Prediction')
plt.tight_layout()
plt.show()
```



Polynomial Regression Comparison

ผลลัพธ์ที่คาดหวัง: - **Degree = 1:** เส้นตรงไม่สามารถจับรูปแบบโค้งได้ → Underfitting (R2 ต่ำ) - **Degree = 2:** เส้นโค้ง Quadratic พอดีกับข้อมูล → Best Fit (R2 สูงสุด) - **Degree = 5:** เส้นคดเคี้ยวตามจุด Noise → Overfitting (R2 บน Training สูง แต่จะต่ำบน Test)

การป้องกัน Overfitting: Polynomial + Regularization

เมื่อใช้ `PolynomialFeatures` degree สูง จำนวน Features จะเพิ่มขึ้นอย่างรวดเร็ว (เช่น 3 Features + degree=3 → 19 Features) ทำให้เสี่ยงต่อ Overfitting อย่างมาก ทางแก้คือใช้ **Pipeline** ที่รวม `PolynomialFeatures` + `StandardScaler` + **Ridge/Lasso** เข้าด้วยกัน

```python
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split

# แบ่งข้อมูล
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Pipeline: Polynomial (degree=3) + Scale + Ridge Regularization
pipeline = Pipeline([
    ('poly', PolynomialFeatures(degree=3, include_bias=False)),
    ('scaler', StandardScaler()),
    ('ridge', Ridge(alpha=10.0))
])

pipeline.fit(X_train, y_train)

r2_train = r2_score(y_train, pipeline.predict(X_train))
r2_test = r2_score(y_test, pipeline.predict(X_test))

print(f"จำนวน Features หลัง Polynomial: {pipeline.named_steps['poly'].n_output_features_}")
print(f"R2 (Train): {r2_train:.4f}")
print(f"R2 (Test) : {r2_test:.4f}")
# หาก Train และ Test ใกล้เคียงกัน → โมเดลไม่ Overfit
```

ผลลัพธ์:

```
จำนวน Features หลัง Polynomial: 3
R2 (Train): 0.9851
R2 (Test) : 0.9851
```

หลักการสำคัญ: - ใช้ `PolynomialFeatures` เพื่อจับรูปแบบ Non-linear - ใช้ `StandardScaler` เพื่อปรับสเกลก่อนทำ Regularization (จำเป็นมาก) - ใช้ `Ridge` เพื่อป้องกัน Overfitting จากจำนวน Features ที่มากเกินไป

สรุปประเด็นสำคัญ

- **Polynomial Regression** ขยายขอบเขตของ Linear Regression ให้จับรูปแบบความสัมพันธ์แบบโค้งได้ โดยการสร้างตัวแปรยกกำลัง (x^2, x^3, ...)
- **การเลือก Degree:** ค่า Degree ต่ำเกินไปทำให้ Underfit, สูงเกินไปทำให้ Overfit — ต้องหาจุดสมดุลด้วย Cross-Validation
- **Pipeline (Poly + Scale + Ridge):** เป็นรูปแบบมาตรฐานในการใช้ Polynomial Regression ในทางปฏิบัติ เพื่อป้องกัน Overfitting

คำถามท้าย Section

1. หากพล็อต Residual Plot ของ Linear Regression (degree=1) พบว่า Residuals มีรูปแบบเป็น “U-shape” ปรากฏการณ์นี้บ่งบอกว่าอะไร และนักศึกษาจะแก้ไขปัญหานี้ได้อย่างไร?
2. เมื่อใช้ `PolynomialFeatures(degree=5)` กับชุดข้อมูลที่มี 10 Features เดิม จำนวน Features ใหม่จะเพิ่มเป็นเท่าใด? เหตุใดจึงต้องใช้ Regularization ร่วมด้วยเสมอ?

☐ สรุปท้ายบท

ในบทนี้ นักศึกษาได้เรียนรู้เกี่ยวกับ: - **Regression:** ใช้ทำนายค่าต่อเนื่อง — คำถามแบบ 'เท่าไหร่?' ใช้ Regression - **Simple/Multiple LR:** Simple ใช้ 1 ตัวแปร, Multiple ใช้หลายตัวแปร — สมการเส้นตรง - **Evaluation Metrics:** MAE, MSE, RMSE วัดขนาด Error; R^2 วัดสัดส่วนที่อธิบายได้ - **Regularization:** Ridge (L2) ลดขนาด Coefficients, Lasso (L1) ตัดตัวแปรที่ไม่สำคัญ - **Polynomial Regression:** ใช้ PolynomialFeatures จับรูปแบบโค้ง ควรใช้คู่กับ Regularization

คำศัพท์สำคัญ: ลักษณะของปัญหา Regression และเมื่อไหร่ที่ควรใช้, ตัวอย่างการประยุกต์ใช้ในบริบทประเทศไทย, รูปแบบทางคณิตศาสตร์เบื้องต้นของปัญหา Regression, การสำรวจข้อมูลสำหรับปัญหา Regression ผ่านชุดข้อมูลจำลอง, รูปแบบของสมการคณิตศาสตร์ (Mathematical Equation), วิธีกำลังสองน้อยที่สุด (Ordinary Least Squares: OLS), การสร้างแบบจำลองด้วย Python และการแสดงผล, ข้อควรระวังและสมมติฐานเบื้องต้น, การตีความสัมประสิทธิ์ (Interpreting Coefficients), สัมประสิทธิ์แบบดิบและแบบมาตรฐาน (Raw vs. Standardized Coefficients), ภาวะร่วมเชิงเส้นพหุ (Multicollinearity), Variance Inflation Factor (VIF), Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Coefficient of Determination (R^2), Adjusted R^2, Regression Diagnostics, LINE Assumptions, Feature Selection, Data Transformation, Regularization, Ridge Regression (L2), Lasso Regression (L1), Polynomial Regression

☐ เอกสารอ้างอิง

- Grus, J. (2019). *Data Science from Scratch* (2nd ed.). O'Reilly.
- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An Introduction to Statistical Learning: with Applications in Python* (2nd ed.). Springer.
- McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.
- VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.

✏ แบบฝึกหัดท้ายบท

☐ ส่วนที่ 1: คำถามทบทวน

คำชี้แจง: *จงเลือกคำตอบที่ถูกต้องที่สุดเพียงข้อเดียว*

1 ปัญหาในข้อใดต่อไปนี้เหมาะสมกับการใช้โมเดล Regression มากที่สุด

A การทำนายว่านักศึกษาจะสำเร็จการศึกษาภายใน 4 ปีหรือไม่ (สำเร็จ/ไม่สำเร็จ)

B การจำแนกประเภทของอีเมลว่าเป็นสแปม (Spam) หรืออีเมลปกติ (Ham)

C การทำนายค่าไฟฟ้ารายเดือนของอาคารเรียนรวม มหาวิทยาลัยขอนแก่น โดยอิงจากอุณหภูมิเฉลี่ย

D การจัดกลุ่มลูกค้าของธนาคารตามพฤติกรรมการใช้จ่าย

▶ แสดงเฉลย

2 ในสมการ Simple Linear Regression $y = \beta_0 + \beta_1 x + \epsilon$ ค่า β_1 (Coefficient) มีความหมายทางสถิติอย่างไร

A ค่าของ y เมื่อ x มีค่าเท่ากับ 0

B อัตราการเปลี่ยนแปลงของ y เมื่อ x เปลี่ยนแปลงไป 1 หน่วย

C ค่าความคลาดเคลื่อนเฉลี่ยของโมเดล

D สัดส่วนความแปรปรวนของ y ที่อธิบายได้ด้วย x

▶ แสดงเฉลย

3 หากชุดข้อมูลที่ใช้ฝึกสอน (Training Data) มีข้อมูลค่าผิดปกติ (Outliers) จำนวนมาก การใช้ Evaluation Metric ในข้อใดจะสะท้อนให้เห็นถึงข้อผิดพลาดของโมเดลได้ชัดเจนที่สุด

A Mean Absolute Error (MAE)

B Mean Squared Error (MSE)

C R^2 Score

D Adjusted R^2

▶ แสดงเฉลย

4 เมื่อทำการวิเคราะห์ Regression Diagnostics โดยการพล็อต Residual Plot (แกน x คือ Predicted Values, แกน y คือ Residuals) หากพบว่าจุดข้อมูลมีการกระจายตัวเป็นรูปกรวย (Funnel Shape) บ่งบอกถึงการละเมิดข้อตกลงเบื้องต้น (Assumption) ในเรื่องใด

A Linearity (ความสัมพันธ์เชิงเส้น)

B Independence of Errors (ความเป็นอิสระของความคลาดเคลื่อน)

C Homoscedasticity (ความแปรปรวนของความคลาดเคลื่อนคงที่)

D Normality of Errors (การแจกแจงปกติของความคลาดเคลื่อน)

▶ แสดงเฉลย

5 ความแตกต่างหลักระหว่างการทำ Regularization แบบ Ridge (L2) และ Lasso (L1) คืออะไร

A Ridge ใช้สำหรับปัญหา Classification ส่วน Lasso ใช้สำหรับ Regression

B Lasso สามารถลดค่าสัมประสิทธิ์ (Coefficient) ของตัวแปรบางตัวให้กลายเป็นศูนย์ได้พอดี จึงช่วยในการคัดเลือกคุณลักษณะ (Feature Selection)

C Ridge มีแนวโน้มที่จะทำให้เกิด Overfitting มากกว่า Lasso

D Lasso คำนวณได้เร็วกว่า Ridge เสมอในทุกชุดข้อมูล

▶ แสดงเฉลย

☐ ส่วนที่ 2: คำถามอัตนัย

คำชี้แจง: *จงอธิบาย วิเคราะห์ หรือเปรียบเทียบ ตามหลักการทางวิทยาการข้อมูล*

1 การประเมินประสิทธิภาพของ Multiple Linear Regression

นักศึกษาคนหนึ่งสร้างโมเดลทำนายราคาบ้านในจังหวัดขอนแก่น โดยเริ่มต้นใช้ตัวแปรต้น (Features) 5 ตัว ได้ค่า R^2 = 0.75 และ Adjusted R^2 = 0.74 ต่อมานักศึกษาได้เพิ่มตัวแปรต้นแบบสุ่ม (Random noise) เข้าไปอีก 50 ตัวแปร พบว่าค่า R^2 เพิ่มขึ้นเป็น 0.82 แต่ค่า Adjusted R^2 กลับลดลงเหลือ 0.61 จงอธิบายสาเหตุทางคณิตศาสตร์และสถิติว่าเหตุใดจึงเกิดปรากฏการณ์นี้ และนักศึกษาควรเลือกใช้ Metric ใดในการตัดสินใจเลือกโมเดล

► แสดงแนวคำตอบ

2 การวิเคราะห์ Residuals ในข้อมูลบริบทสาธารณสุข

พิจารณาการสร้างโมเดลทำนายจำนวนผู้ป่วยนอก (Outpatients) รายวันของโรงพยาบาลศรีนครินทร์ โดยใช้ตัวแปรเช่น อุณหภูมิ ปริมาณฝุ่น PM2.5 และวันในสัปดาห์ หลังจากสร้างโมเดล Linear Regression และตรวจสอบ Residuals พบว่าค่า Error ของเมื่อวานมีความสัมพันธ์เชิงบวกกับค่า Error ของวันนี้อย่างมีนัยสำคัญ (High Autocorrelation) ปรากฏการณ์นี้บ่งบอกถึงอะไร และจะมีวิธีการปรับปรุงโมเดลอย่างไร

► แสดงแนวคำตอบ

3 การเลือกใช้เทคนิค Regularization

ทีมวิจัยการเกษตรต้องการสร้างโมเดลทำนายผลผลิตอ้อย (ตันต่อไร่) โดยมีข้อมูลจากการเก็บตัวอย่างเพียง 100 แปลง (n = 100) แต่มีการติดตั้งเซนเซอร์และเก็บข้อมูลตัวแปรทางสภาพแวดล้อมและดินถึง 250 ตัวแปร (p = 250) ในสถานการณ์ที่ $p > n$ เช่นนี้ นักวิทยาศาสตร์ข้อมูลควรเลือกใช้เทคนิค Ridge Regression หรือ Lasso Regression? จงอธิบายเหตุผลประกอบ

► แสดงแนวคำตอบ

ส่วนที่ 3: แบบฝึกหัด Coding

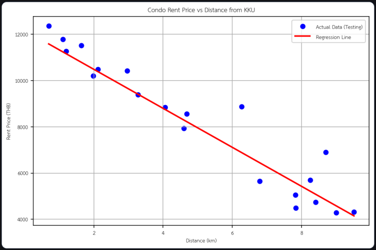
คำชี้แจง: *ให้นักศึกษาเขียนโปรแกรมภาษา Python โดยใช้ไลบรารี* `scikit-learn`, `pandas` *และ* `numpy` *เพื่อแก้ปัญหาต่อไปนี้*

ข้อ 1. Simple Linear Regression และการประเมินผล โจทย์: สมมติว่านักศึกษาต้องการศึกษาความสัมพันธ์ระหว่าง “ระยะทางจากมหาวิทยาลัยขอนแก่น (กิโลเมตร)” กับ “ราคาเช่าคอนโดมิเนียมรายเดือน (บาท)” 1. สร้างชุดข้อมูลจำลอง (Synthetic Data) จำนวน 100 ข้อมูล 2. แบ่งข้อมูลเป็น Training Set (80%) และ Testing Set (20%) 3. สร้างและฝึกสอนโมเดล Simple Linear Regression 4. ทำนายผลบน Testing Set และคำนวณค่า Mean Absolute Error (MAE) และ R^2 Score 5. พล็อตจุดข้อมูล Testing Set พร้อมกับเส้นตรงที่โมเดลทำนายได้ (Regression Line)

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

# 1. สร้างชุดข้อมูลจำลอง
np.random.seed(42)
# ระยะทาง 0.5 ถึง 10 กิโลเมตร
X = np.random.uniform(0.5, 10.0, 100).reshape(-1, 1)
# ราคาเช่าเริ่มต้น 12000 บาท ลดลง 800 บาทต่อกิโลเมตร พร้อมบวกค่า Noise
y = 12000 - 800 * X.flatten() + np.random.normal(0, 1000, 100)

# 2. แบ่งข้อมูล Train (80%) และ Test (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. สร้างและฝึกสอนโมเดล Linear Regression
model = LinearRegression()
model.fit(X_train, y_train)

# 4. ทำนายผลและประเมินประสิทธิภาพ
y_pred = model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"สมการเชิงเส้น: Price = {model.intercept_:.2f} + ({model.coef_[0]:.2f} * Distance)")
print(f"Mean Absolute Error (MAE): {mae:.2f} บาท")
print(f"R-squared (R2): {r2:.4f}")

# 5. พล็อตจุดข้อมูลและเส้น Regression Line
plt.figure(figsize=(8, 5))
plt.scatter(X_test, y_test, color='blue', label='Actual Data (Testing)')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Regression Line')
plt.title('Condo Rent Price vs Distance from KKU')
plt.xlabel('Distance (km)')
plt.ylabel('Rent Price (THB)')
plt.legend()
plt.grid(True)
plt.show()
```

ผลลัพธ์:

```
สมการเชิงเส้น: Price = 12164.04 + (-842.25 * Distance)
Mean Absolute Error (MAE): 591.34 บาท
R-squared (R2): 0.9112
```



Exercise 1: Simple LR

ข้อ 2. Polynomial Features และ Ridge Regression เพื่อป้องกัน Overfitting โจทย์: ในการทำนายปริมาณการใช้ไฟฟ้า (กิโลวัตต์-ชั่วโมง) ของโรงงานอุตสาหกรรมแห่งหนึ่ง มีตัวแปรต้น 3 ตัว คือ อุณหภูมิแวดล้อม (X_1), จำนวนพนักงานที่ปฏิบัติงาน (X_2), และชั่วโมงการทำงานของเครื่องจักร (X_3) เนื่องจากความสัมพันธ์อาจไม่เป็นเส้นตรง นักศึกษาจึงตัดสินใจใช้ `PolynomialFeatures` (degree=3) ซึ่งสร้างตัวแปรใหม่จำนวนมากจนอาจเกิด Overfitting 1. ให้ใช้ `Pipeline` ในการเชื่อมต่อ `PolynomialFeatures` และ `Ridge` Regression 2. ฝึกสอนโมเดลด้วย `alpha=10.0` 3. เปรียบเทียบค่า R^2 ระหว่าง Training Set และ Testing Set เพื่อตรวจสอบว่าโมเดลยังมีปัญหา Overfitting หรือไม่

```python
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import Ridge
from sklearn.pipeline import Pipeline
from sklearn.metrics import r2_score

# สร้างชุดข้อมูลจำลอง (ตัวแปรต้น 3 ตัว, 150 ข้อมูล)
np.random.seed(99)
X = np.random.rand(150, 3) * 100  # X1: Temp, X2: Workers, X3: Machine Hours
# สร้างความสัมพันธ์แบบ Non-linear (มีกำลังสอง และผลคูณระหว่างตัวแปร)
y = 50 + 2.5*X[:, 0]**2 - 1.5*X[:, 1] + 3.0*(X[:, 0]*X[:, 2]) + np.random.normal(0, 500, 150)

# แบ่งข้อมูล
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=99)

# 1. สร้าง Pipeline: ขยาย Feature → ปรับสเกล → Ridge Regression
# หมายเหตุ: การใช้ Regularization (Ridge) จำเป็นต้องทำ Standard Scaling เสมอ
pipeline = Pipeline([
    ('poly', PolynomialFeatures(degree=3, include_bias=False)),
    ('scaler', StandardScaler()),
    ('ridge', Ridge(alpha=10.0)) # 2. กำหนด alpha เป็น 10.0 เพื่อควบคุม Overfitting
])

# ฝึกสอนโมเดล
pipeline.fit(X_train, y_train)

# 3. ทำนายผลและเปรียบเทียบ R2
y_train_pred = pipeline.predict(X_train)
y_test_pred = pipeline.predict(X_test)

r2_train = r2_score(y_train, y_train_pred)
r2_test = r2_score(y_test, y_test_pred)

print(f"จำนวน Features ดั้งเดิม: {X_train.shape[1]}")
print(f"จำนวน Features หลังทำ Polynomial (degree=3): {pipeline.named_steps['poly'].transform(X_train).sh
print("-" * 40)
print(f"R-squared บน Training Set: {r2_train:.4f}")
print(f"R-squared บน Testing Set: {r2_test:.4f}")

# คำอธิบายเพิ่มเติมสำหรับนักศึกษา:
# หาก R2 ฝั่ง Train สูงมาก (เช่น 0.99) แต่ Test ต่ำมาก (เช่น 0.50) แสดงว่าเกิด Overfitting
# การปรับค่า alpha ใน Ridge ให้สูงขึ้น จะช่วยดึงให้ R2 ของ Train และ Test ใกล้เคียงกันมากขึ้น
```

ผลลัพธ์:

```
จำนวน Features ดั้งเดิม: 3
จำนวน Features หลังทำ Polynomial (degree=3): 19
----------------------------------------
R-squared บน Training Set: 0.9965
R-squared บน Testing Set: 0.9963
```

← บทที่ 7: แนะนำ Machine Learning

บทที่ 9: Classification — การจำแนกประเภท →
